

sDDF Design

Design, Implementation and Evaluation of the seL4 Device Driver Framework

Gernot Heiser, Peter Chubb, Alex Brown, Tristan Clinton-Muehr,
Courtney Darville, Alwin Joshy, Craig McLaughlin, Bill Nguyen,
Lesley Rossouw, Lucy Fletcher, Julia Vassiliki, Ivan Velickovic,
Jade Zhou

gernot@unsw.edu.au

Release 0.7, 2026-08-19

Abstract

This is a work-in-progress report that documents the design of a high-performance device driver framework for seL4, including the structure and interfaces of compliant drivers, and presents some preliminary evaluation.

The document is intentionally explicit about the assumptions it makes on hardware (the *device model*) and the structure it prescribes on device drivers (the *driver model*). This is to facilitate exploring formal specification and, eventually, verification of device drivers.

Consequently, besides specifying how drivers and their interfaces are structured, the document also serves to define the context for the *Pancake* project that develops a programming language for verifiable device drivers. As such, the report serves as an informal interface document between the Pancake team and systems researchers (and thus explains many things systems people take for granted).

Contents

List of Figures	v
List of Listings	vi
Acronyms	vii
1 Aims of the sDDF	1
2 Threat Model	2
3 Device Model	3
3.1 Device interfaces	4
3.2 Device interface protocol	5
3.3 Device operation	5
3.4 Comments	6
4 Drivers and Their Software Interface	8
4.1 Overview	8
4.2 Transport layer	10
4.2.1 Memory regions	10
4.2.2 Data region	11
4.2.3 Metadata regions	12
4.2.4 Device metadata region	14
4.3 Component models	15
4.3.1 Driver	15
4.3.2 Virtualiser	16
4.3.3 Observations	17
4.4 Synchronisation	18
4.4.1 Active driver-thread model	18
4.4.2 Passive driver-thread model	21
4.4.3 Two-component driver	22
4.4.4 Time-triggered architectures	23
4.4.5 Discussion	23
5 Device Classes	25
5.1 Network	25
5.1.1 Main properties	25
5.1.2 Ethernet	26
5.1.3 Status	26

5.2	Asynchronous serial busses	26
5.2.1	UART	26
5.2.2	USB	27
5.3	Synchronous serial busses	27
5.3.1	I2C host	27
5.3.2	SPI	31
5.4	Storage	31
5.4.1	Main properties	31
5.4.2	Details	32
5.4.3	Status	34
5.5	Sound	35
5.5.1	Main properties	35
5.5.2	Message Protocol	35
5.5.3	Playback	36
5.5.4	Recording	37
5.5.5	Sound Virtualisers	37
5.5.6	Status	37
5.6	Display	38
5.7	Video Capture	38
5.8	Pin Control	38
5.8.1	Terminology	38
5.8.2	Main properties	38
5.8.3	Message Protocol	38
5.8.4	Status	39
5.9	Sensors, PWM, GPIO, etc	39
5.9.1	Sensors	39
5.9.2	GPIOs	40
5.9.3	PWM	40
5.9.4	Status	40
6	Device Discovery	41
7	Leveraging Linux	42
7.1	Component development	42
7.2	Legacy driver reuse	44
8	Security Analysis	45
8.1	Trusted components	45
8.2	Verifying components	45
8.3	Verifying component interactions	46
9	Implementation Status	47
10	Performance	48
10.1	Performance evaluation setup	48
10.2	Performance results	49
10.2.1	Simplified system, CAMkES, Linux	49
10.2.2	Cost of modularity and security	50
10.2.3	Multicore	52
10.3	Discussion	53

11 Conclusions	55
Bibliography	56
A OS Abstraction	58
A.1 Microkit Mapping	58
B Overview of Changes	60
B.1 Changes For Release 0.7 of 2025-03-21	60
B.2 Changes Since Release 0.6 of 2025-03-21	60
B.2.1 New device classes	60
B.3 Changes Since Release 0.4 of 2024-03-27	60
B.3.1 OS abstraction	60
B.3.2 More device classes	60
B.4 Changes Since Release 0.2 of 2022-10-03	60
B.4.1 More device classes	60
B.4.2 Mandatory virtualisers, cache maintenance	61
B.4.3 Completely separated Tx and Rx paths	61
B.4.4 Clarified terminology	61
B.4.5 Linux-based component development and legacy re-use	61
B.4.6 Performance evaluation	61
B.4.7 Changes resulting from evaluation	62

List of Figures

3.1	Device model.	3
4.1	High-level structure of I/O systems on seL4, using network terminology.	8
4.2	Memory regions shared between software components and the device.	10
4.3	Tx virtualiser-driver transport layer, showing the <i>metadata</i> and <i>data regions</i>	12
4.4	Active (left) vs passive (right) driver-thread model.	18
4.5	Mode switches for the active driver-thread model compared to Linux.	19
5.1	Modularised design of a networking architecture.	25
5.2	Memory regions shared between software components and the device.	28
5.3	Storage device transport layer.	31
5.4	TX prebuffering	37
7.1	Linux driver re-use through UIO.	43
10.1	Evaluation setup for sDDF (top) and Linux (bottom).	48
10.2	sDDF networking performance compared to other systems running single-core.	49
10.3	Performance of differently modularised sDDF systems compared to Linux.	50
10.4	Comparing per-packet processing costs of the four sDDF systems.	51
10.5	Single-core sDDF performance with single- and multicore configurations.	51
10.6	Single- and two-core sDDF performance compared to 2-core Linux.	52

List of Listings

4.1	Control region queue data structures.	13
4.2	Control-region queue management.	14
4.3	Typical network device queue data structures.	15
4.4	Device queue management (simplified).	16
4.5	Ethernet driver pseudocode.	20
5.1	I2C command structure.	29
5.2	Storage request and response queue data structures.	33
5.3	Storage information region data structure.	34
5.4	Sound definitions	36
7.1	Example UIO stub driver command-line configurations.	42
7.2	Example UIO node in device tree.	42
7.3	Example of Linux usermode driver helper.	44
A.1	sDDF interface functions.	59

Acronyms

API application programming interface 21

CAMkES component architecture for microkernel-based embedded systems, a user-level framework for developing on top of seL4 49, 50, 53, 55

CD-ROM compact-disk read-only memory 32

CPU central processing unit 1, 3, 5, 14, 17, 21, 49, 50, 52, 53

CSpace capability address space 9

DMA direct memory access 3, 4, 5, 6, 9, 10, 11, 12, 13, 14, 17, 32, 36, 42, 45

I/O input/output 4, 5, 9, 11, 13, 16, 17, 21, 26, 31, 32, 37, 44, 61

I2C Inter-Integrated Circuit (serial communication bus) 27, 28, 29, 30, 47

IOMMU input/output memory management unit 5, 16, 17, 45

IRQ interrupt request 6, 18, 19, 21, 22, 23, 24, 43

MMU memory management unit 4, 5

NIC network interface card 9, 15, 21, 22, 26, 46, 48, 50

OS operating system 1, 8, 9, 16, 18, 58

PCM pulse-code modulation 35, 36, 37

PCM physical layer 48

PD Protection Domain: a virtual address space plus a thread of control. A PD roughly corresponds to a Unix process. 58

PPC seL4 protected procedure call 9, 39, 40, 58

Rq request 16, 17, 31, 32

Rs response 31, 32

Rx receive iv, 9, 10, 11, 13, 15, 16, 17, 25, 26, 31, 52, 61

SC scheduling context 18, 21, 22, 23

sDDF seL4 Device Driver Framework v, 1, 9, 21, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 58, 60

TCB trusted computing base 2

Tx transmit iv, 9, 10, 11, 12, 13, 14, 15, 16, 17, 19, 25, 26, 31, 52, 61

UIO Linux userspace I/O v, vi, 42, 43, 44, 61

VirtIO Linux I/O virtualisation framework 37

VM virtual machine 34, 42, 43, 44, 55

VM virtual-machine monitor 43, 44

VSpace virtual address space 9

Chapter 1

Aims of the sDDF

Every operating system (OS) needs device drivers, and OSes based on the seL4 microkernel [Klein et al., 2009; seL4 Foundation, 2021] are no exception.

The seL4 Device Driver Framework (sDDF) provides libraries, interface specifications and tools for writing/porting device drivers to run as native, isolated components on seL4. Specifically, the sDDF design aims to:

- support a wide class of devices, including network, USB, graphics, storage, serial ports etc;
- achieve performance comparable to Linux in-kernel drivers, as measured by throughput, latency and CPU load, at least for latency-sensitive high-throughput devices (network interface cards and USB);
- be robust against defined threats;
- support sharing of devices between multiple clients, which might be seL4-native components or virtual machines;
- provide strong *separation of concerns*;
- eventually enable the formal verification of its components.

The sDDF assumes a general device model that should enable formal reasoning. It defines a device driver model that is appropriate for seL4 and aims to simplify driver implementation, eliminate common causes of driver bugs and aid formal verification. It is based on an asynchronous, zero-copy transport layer that minimises overheads while keeping driver interfaces simple.

For the time being we focus on defining a driver model and its control and data interfaces to the driver's clients, leaving other required functionality, such as device discovery and device initialisation for future work.

Chapter 2

Threat Model

The ultimate aim is a system whose whole *trusted computing base* (TCB) is verified to be *trustworthy*. This requires the TCB to be minimised. An implication is that some drivers will be part of the TCB and thus trusted, while others (the majority) of drivers are not. It also means that the driver framework itself is trusted.

We assume that an attacker is able to compromise and fully control any component that is not part of the TCB. Specifically this means that **the attacker may**:

- execute arbitrary instructions in the address space of any untrusted driver;
- execute arbitrary instructions in the address space of any untrusted client;
- arbitrarily change the data on untrusted external media (network or physically insecure storage).

However, **the attacker cannot**:

- compromise a trusted component (driver, virtualiser);
- compromise a trusted client;
- compromise the driver framework itself.

We assume that trusted clients or servers dealing with untrusted entities use encryption protocols, such as TLS, to ensure confidentiality and integrity of data, even if using untrusted components. If a client only employs trusted components, the system must guarantee confidentiality, integrity and availability.

The threat model implies that it must be feasible to formally verify all trusted components (eventually).

Chapter 3

Device Model

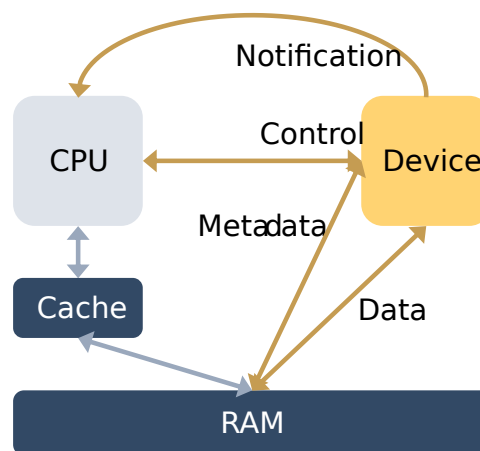


Figure 3.1: Device model.

We view a device as a state machine (implemented in hardware) with four interfaces, indicated in Figure 3.1:

the control interface specifies how the software running on the CPU, i.e. the *device driver*, issues commands to the device, and how the device reports status information back to the driver;

the notification interface lets the device alert the driver to a state change;

the data interface transports data between software and the device;

the metadata interface specifies data locations and is referenced by the control interface.

The notification interface is one-way (device to driver) while the control interface is two-way. The data interface can be one-way: for a pure input device (microphones, cameras, read-only storage media), data only travels from the device to the driver, while for a pure output device (audio output, display), data only travels from the driver to the device; other devices (network, storage) have two-way data interfaces. The metadata interface is two-way, as for both input and output channels it is updated by both sides.

The model is presently somewhat simplified, in that it does not take into account hot-plugging/unplugging (important e.g. for USB devices), nor the possibility of separate DMA controllers.

3.1 Device interfaces

The **control interface** is memory-mapped: The driver accesses the interface by reading from or writing to specific physical addresses, *device registers*, which are mapped (by the memory management unit, MMU) into the driver's virtual address space.¹

These device registers do not behave like regular memory! Specifically:

- a write to a device register followed by a read from the same register might *not* return the value written (and may result in an error condition);
- the sequence of reads or writes matters: the outcome may change if register accesses are reordered;
- the granularity of device register accesses matters: a byte-sized write to a register address followed by three byte-sized writes to subsequent addresses might not have the same effect as a 32-bit write to the same address.
- multiple reads or writes to the *same* address are all significant, and can have a user-visible effect, even if the values read/written are identical.

Among others, these properties require that device registers are mapped uncached, and that software declares them *volatile*. The correct granularity and sequence of device register accesses are defined by the device interface protocol.

The **notification interface** is an interrupt (which the seL4 kernel converts into signalling a *notification* object). It instructs the driver to use the control interface to determine what is being notified. Interrupts can signal a number of conditions:

- **transmit completed:** the device has completed an output (data write) operation and will no longer access the relevant output data;
- **data available:** the device has completed an input (data read) operation, will no longer access the respective buffer and software can now safely access the data;
- **other state changes:** the device state has changed in some other way, which includes such events as disk spin up completed, network cable inserted, firmware download completed;
- **error:** some failure occurred (which includes a device losing power or being disconnected).

The **data** and **metadata interfaces** use actual memory (called *direct memory access*, DMA). This means that the device accesses the memory concurrently with software, similar to a separate processor core. This has a number of implications:

- DMA generally bypasses the cache. While some processors (specifically the x86 architecture) hides this fact by ensuring the cache remains coherent with DMA memory, others (specifically Arm) do not, requiring the use of memory barriers for consistency, and the use of cache-management operations (cache flush or invalidate).
- Software must not read DMA memory while an input is in progress, and must not write DMA memory while an output is in progress. In fact, because reads and writes affect

¹On the x86 architecture some legacy devices, specifically serial ports, are not memory mapped and require I/O port instructions to control. These devices process small amounts of data (have no data interface) and are not performance-critical. They will be handled in an ad-hoc manner.

the cache, it is advisable not to perform any reads or writes to a DMA buffer during I/O to that buffer.

- Speculative reads (aka. pre-fetching) on DMA memory may happen while the device is writing input data, requiring extra cache management on input buffers.
- Addresses used for DMA are either not mapped (i.e., they are physical addresses), or are mapped via a different MMU, the IOMMU, from the addresses used by the CPU.
- For the *metadata region*, explicit cache management is usually not worthwhile, meaning that the CPU should map this memory *uncached* (unless the hardware maintains cache coherency, as on x86).

Simple devices (serial port, timers) do not usually use DMA. We can treat them as the special case of an empty data and metadata interface.

3.2 Device interface protocol

Software access to device registers (writes and potentially reads) trigger state transitions in the device; a device state transition may result in a software-visible change in a device register.

The device interface protocol specifies the addresses, sizes and semantics (i.e. state transitions triggered by access) of device registers. It also specifies some ordering conditions on accesses (reads or writes). It may specify timing conditions on accesses.

Examples of timing conditions are:

- access B must happen no earlier than x microseconds after access A ;
- after access A , the driver must poll (read) register b until it is non-zero, before performing access C .

For the case that the protocol specifies minimal delays between accesses, we can assume that there are only a small number of such delay values, which are calibrated at device setup time and are abstracted by delay functions defined outside the driver proper.

3.3 Device operation

A sequence of state transitions may move the device into a state where it performs **output**, by reading DMA memory. Specifically it will follow references, supplied by the control interface, to descriptors in DMA memory (the metadata interface), and from there to the actual data (data interface). Addresses for DMA have to be translated to the device address space before being given to the device. The device may use physical addresses or its own virtual address space (mapped by the IOMMU).

The completion of the output operation results in a state transition that triggers the notification interface (i.e. an interrupt); the driver then needs to use the control interface to determine the reason for the interrupt (i.e. output completed).

A sequence of state transitions may move the device to a state where it can provide **input** to DMA buffers provided by the driver via the metadata interface, which in turn was referenced by the control interface. To software, the commencement of input is non-deterministic (it is

determined by the environment). The completion of input results in a state transition that triggers the notification interface, the driver then needs to use the control interface to determine the reason for the interrupt (i.e. data available).

Once an interrupt has triggered, the hardware disables the interrupt (and lower-priority ones), preventing it from re-occurring if it is a level-triggered interrupt. The kernel immediately masks that specific interrupt (preventing it from triggering again) and then acknowledges it (which re-enables all the lower priority ones). When the driver *acknowledges* the interrupt to the kernel, the kernel unmask the interrupt, after which the driver can wait for the next interrupt.

A single interrupt may signal multiple completions (this is called *interrupt coalescing*). The driver needs to process all pending completions prior to acknowledging the interrupt. Depending on the device, the driver may also have to clear interrupt conditions in the device registers to prevent the interrupt re-triggering.

Completions can continue to occur while interrupts are masked, which implies that upon acknowledging, the interrupt may immediately trigger again, resulting in the driver immediately receiving a new notification as soon as it finished processing the present one. In order to minimise the number of kernel entries, the driver should, before acknowledging the IRQ, use the device's control interface to check for pending interrupts. After processing all pending events, the driver clears the device's event register. (Note: this is a performance optimisation, not a correctness issue.)

If interrupts are masked for too long, or software is too slow to process input data, the device will run out of free DMA buffers for depositing input data; in this case data will be lost (the device drops packets). This creates an automatic flow control of inputs, and can be used to prevent overloading the system (called *rate limiting* the device).

Network packets can be lost for external reasons too, e.g. in a router between source and destination, or by interference on a wireless link. Higher-level network protocols (e.g. TCP) handle packet loss using acknowledgement and re-transmission.

We ignore error conditions for now, but note that dropping of packets is not considered an error condition.

Devices that can be plugged into or removed from a running system, such as USB devices or SD cards, add further complications which we ignore for now. Support for "hotplugging" will be covered in ??.

3.4 Comments

The device interface protocol is defined by the hardware, and is thus not under our control. To enable formal reasoning about driver correctness, the device protocol will have to be formalised.

Unfortunately, these protocols are usually specified in manufacturers' data sheets that are highly informal, typically vague, and frequently wrong. Errors in data sheets arise from device implementation bugs as well as a manufacturer-internal miscommunication between hardware designers and documenters. Even worse, there are many cases where not even a data sheet is available, and the interface is reverse-engineered from a Linux driver implementation.

Buggy device interface specs will inherently lead to buggy drivers. Unfortunately, this is not something we can address for now, we are at the mercy of what is available. However, it is a

problem *every* driver developer faces, whether or not they employ formal reasoning.

But we do have the opportunity to at least eliminate all other driver bugs, which are the majority [Ryzhyk et al., 2009]. And should we succeed in verifying realistic drivers, we can offer a good value proposition to hardware manufacturers, and may be able to tackle the specification problem in partnership with device IP producers.

Chapter 4

Drivers and Their Software Interface

Our driver model strongly reflects the aim of separation of concerns: *the sole purpose of a device driver is hardware abstraction*. The driver translates a hardware-defined and -specific device protocol into a hardware-independent (but OS-specific) *device-class protocol*. In other words, there is a single software-side driver interface specification for each device class (network, USB, storage, etc). This interface is designed to support a lightweight, low-overhead translation, at least for performance-sensitive (high-throughput) devices.

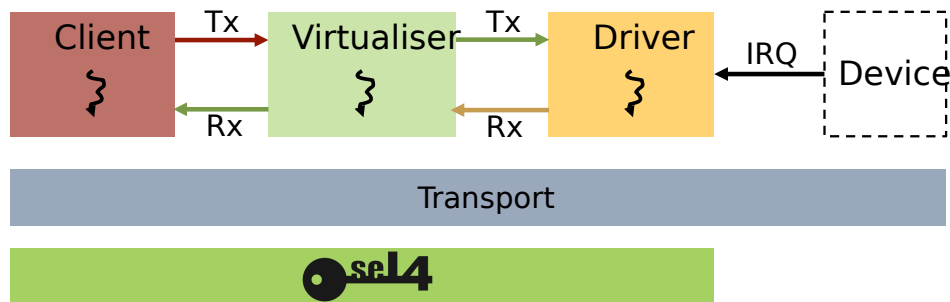


Figure 4.1: High-level structure of I/O systems on seL4, using network terminology. Coloured boxes are software components, grey indicates a shared memory region.

4.1 Overview

In our design we utilise the flexibility afforded by a clean-slate design, where we can control the structure of the drivers as well as their interface to the rest of the system’s software. Specifically we use this flexibility to define a structure that eliminates unnecessary complexity in driver implementations and thus makes it easier to write performant yet correct drivers (with the intent to verify them eventually).

Our design is based on our prior work [Leslie et al., 2005], which established that, using a zero-copy transport layer with shared queues and asynchronous communication, usermode network drivers can deliver performance competitive with in-kernel drivers. It also incorporates what we have learned from the later Dingo work [Ryzhyk et al., 2009], which proposed a driver model for Linux that would eliminate many common driver faults.

There are two general classes of devices that we deal with:

Simple devices, such as sensors, produce or consume simple (typically one-word) data items, and are not shared between multiple clients. Some low-bandwidth communication devices also fall into this category. The drivers of these devices are invoked by PPC (seL4 protected procedure call) to pass data in or out (meaning they inherently use a *passive driver thread* model, as presented in Section 4.4.2). Due to their simplicity this kind of devices needs no further elaboration here. Section 5.9 describes such device classes.

Devices that deal with bulk data, (network, block, video, etc) use queues and shared data regions. The rest of Chapter 4 deals only with bulk-data driver interfaces.

Figure 4.1 shows the high-level logical structure of bulk-data device interfaces in the sDDF. Each coloured box represents a user-level process that is encapsulated by seL4 and only able to communicate via defined interfaces. The high-level view has three components:

Device driver: Interfaces with the device hardware (NIC, flash, ...), receives interrupts (as seL4 *notifications*), and transmits raw data between the device and the client;

Client: Is the producer/consumer of the I/O data handled by the device. In many cases the “Client” is actually an OS server implementing a higher-level abstraction for applications, but this is not relevant to the sDDF.

Virtualiser: Allows a single device to be shared between one or more Clients. Where there are multiple clients, the Virtualiser presents the Client with an illusion of exclusively owning the device. Each control path has its own virtualiser (see Section 4.2.1).

The components are connected by simple control and notification interfaces. The control interfaces refer to a metadata and data interface that is provided by the *transport layer*. The Driver- and Client-side interfaces of the Virtualiser look almost the same, except the Driver-side uses I/O-space addresses, whereas the Client-side uses offsets into the Data region.

The Client-Driver interface traditionally uses somewhat different terminology for different device classes (e.g. network vs storage). For now we use the terminology used for network devices, referring to the output operation as *transmit* (Tx) and the input operation as *receive* (Rx). We will discuss interfaces for different device classes in Chapter 5.

Each of the above processes (i) consists of (at least) one seL4 *virtual address space* (VSpace) with access rights defined by its *capability space* (CSpace), a thread, and scheduling parameters. The latter consist of a priority, P_i , and a (possibly null) scheduling context, S_i . A scheduling context has two relevant parameters, a *budget*, C_i and a *period*, T_i , where $C_i \leq T_i$.

For the rest of this document, unless explicitly stated otherwise, we will use the term Client to refer to the whichever software is interfacing to the driver via the virtualiser, keeping in mind that in a particular configuration this could be a component implementing an OS abstraction, a copier, or a directly-connected application.

In fact, this reflects a flexibility enabled by our design based on strict separation of concerns: components such as copiers can be transparently added or removed depending on the requirements of the specific system. Similarly, issues such as mapping of virtual (Client) addresses to device I/O addresses, or dynamic re-mapping of DMA regions, are of no concern to the Driver.

4.2 Transport layer

The transport layer consists of a number of shared memory regions, data structures in those regions, and access protocols. It is designed to minimise software overheads by minimising (ideally eliminating) copying of data.

4.2.1 Memory regions

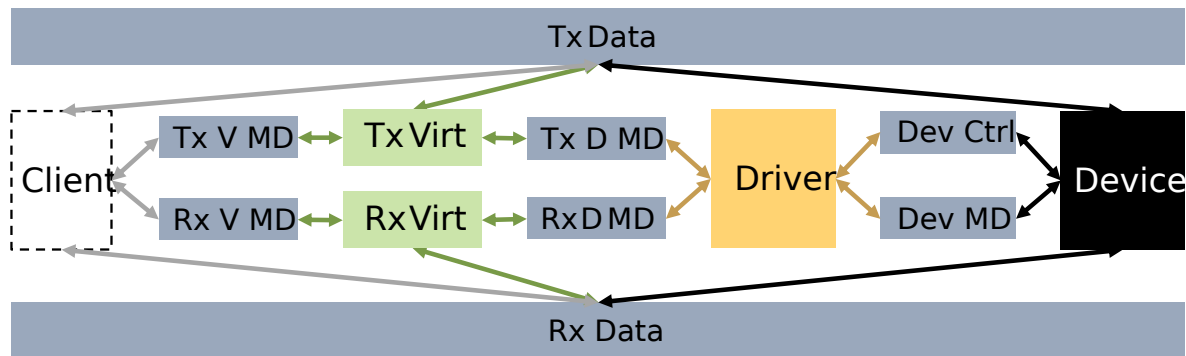


Figure 4.2: Memory regions (grey) shared between software components and the device. “MD” represents metadata regions of the device (Dev), driver (D) and virtualisers (Virt).

There are three types of memory regions, as shown in Figure 4.2:

1. a **device control region** (Dev Ctrl) shared between driver and device
2. multiple **metadata regions** (MD) shared between driver and device, and similar regions shared between components
3. a **data region** shared between client, virtualisers and device.

Note that while the driver holds *references* into the data region, it only passes these between the virtualiser and the device; the driver never needs to access the actual data transferred. The data region is therefore not mapped into the driver’s address space, reducing the trust required in the driver. The driver only needs access to the metadata region.

Also, for devices (e.g. networking) having separate Tx and Rx control paths, the data region will generally be split into separate Tx and Rx regions, as indicated in Figure 4.2.

Driver-device interface

The *device control region* is a set of memory-mapped device registers used to change and inquire the device state; they constitute the primary mechanism for software to interact with hardware. The set of device registers, and thus the size of this region, is hardware-defined, and typically fits on a single page. Multiple devices may share a single page of device registers, in which case the drivers must take care not to interfere with each other.

On architectures that do not ensure cache consistency with DMA, we map the device control region *uncached* in the driver’s address space. This avoids any need for cache management by the driver (without significant performance penalty).

For simple devices, such as serial ports, the device control region constitutes the complete hardware-software interface. Interaction happens by the driver reading or writing individual registers (which can be byte- or word-size).

For devices processing bulk data, there is a *metadata region* (Dev MD), which is of flexible size but usually small (few pages), located in RAM. It contains data structures referencing the data buffers (which transfer the actual input or output data). Typically these are circular queues (“ring buffers”). While the format of those data structures is hardware-defined (part of the device protocol), their size is usually software-defined. The driver uses the device control interface to inform the device of the size and location of the metadata structures.

References to the metadata region (whether passed through the device-control interface or internal pointers in the metadata region) are *I/O space addresses*.

The device accesses the metadata region by DMA. The region is therefore also *mapped uncached* in the driver’s address space, and no cache management is required by the driver.

Virtualiser-driver interface

The driver and virtualiser interface via a separate, shared metadata region. To distinguish it from other MD regions, we refer to this as the *driver metadata* (D MD) region. For device classes featuring multiple (e.g. Tx and Rx) virtualisers, there is a separate metadata region for each virtualiser (Tx D MD and Rx D MD).

As a general rule, the driver metadata regions are regular memory, i.e. *mapped cached* in the driver (as well as the virtualisers). References into the data region from the metadata region use *I/O space addresses*.

For drivers that do not do DMA (e.g. our current serial drivers), the metadata region contains the data to be transferred; they do not have a separate data region.

Client-virtualiser interface

This interface, the *virtualiser metadata region*, again consists of one region per virtualiser. Obviously, each client has its own virtualiser metadata region(s). The region is regular memory (mapped cached) and looks identical to the driver metadata region, *except* that references into the data region are *offsets relative to the start of the data region*. Generally, each client will have its own data region(s). It is part of the virtualiser’s job to translate the offsets into the per-client data regions into *I/O space addresses* for the driver, adding the offset to the corresponding base *I/O space address* of each client’s data region. At least for now we require client data regions to be contiguous area in the physical address space.

4.2.2 Data region

The layout of the data region(s) depend on the device class. In general it contains a set of I/O buffers that are referenced by the various metadata regions. In many cases (e.g. networking) the data region is simply an array of equal-sized I/O buffers.

For some device classes (e.g. networking) the data region can be split into separate regions for input and output.

As mentioned above, references to the data region on the client side of the virtualiser are represented as offsets relative to the start of the region, while the driver side uses I/O-space addresses. Translating between those references is one of the duties of the virtualisers.

In general, each I/O buffer is in one of the following states:

1. **source-owned and in use:** the buffer is not referenced by a *metadata region* (but referenced by source-internal data structures), it is waiting for, or is in the process of, being filled with data;
2. **destination-owned and idle:** the buffer is referenced by a *metadata region*, ready to be collected by the destination, it *contains valid data*;
3. **destination-owned and in use:** the buffer is not referenced by a *metadata region* (but referenced by destination-internal data structures), it may be waiting for or in the process of being used by the destination;
4. **source-owned and idle:** the buffer is referenced by a *metadata region*, ready to be collected by the source, it *contains no useful data*.

Here “source” stands for the component that produces data: For output that would be the component to the left (client side) of the region as per Figure 4.1, while “destination” would be the component to the right (device side) of the region. For input, source refers to the device-side and destination to the client-side component.

Ownership is the exclusive right to access a buffer, as well as the descriptor that references it in its respective metadata region. In other words, if a buffer is owned by a component, no other component may access the buffer nor the queue entry that references the buffer.

For device classes that have separate input and output data regions, a particular DMA buffer is only ever used for either output or input: An output buffer is filled by the client, consumed by the device, and then handed back as free to the client. Similarly, an input buffer is filled by the device, consumed by the client, and eventually returned as free to the device.

While the client may change a buffer’s direction (e.g. receiving a packet from the network, changing some of its content and sending it back to the network), this happens outside the framework (and may not be advisable as it will significantly complicate buffer management).

4.2.3 Metadata regions

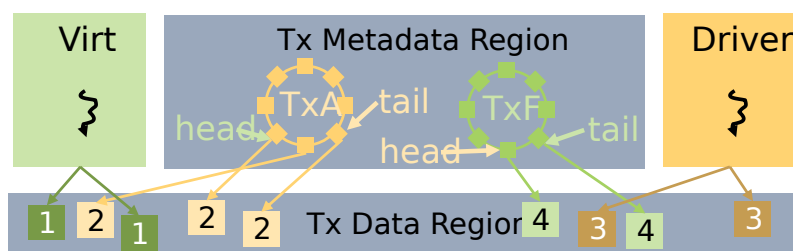


Figure 4.3: Tx virtualiser-driver transport layer, showing the *metadata* and *data regions*. The numbers in the buffers in the data region indicate the buffer state as defined in Section 4.2.2.

Figure 4.3 shows the structure of the transport layer interfacing the virtualiser and the driver, again using a Tx path as an example.

The metadata region uses *lockless, bounded, single-producer, single-consumer queues*. These are implemented as what is commonly referred to as “ring buffers”. To avoid confusion, we will not use this terminology, and only refer to them as “queues”. We reserve the term “buffer” for the DMA-able data buffers.

The Tx metadata structure consists of two fixed-size queues, *transmit active* (Tx A) of output data to be handed to the device, and *transmit free* (Tx F) of buffers ready to be re-used. The

TxA queue references all buffers that are destination-owned and idle, the TxF queue references all buffers that are source-owned and idle.

The Rx path has an equivalent structure, with two queues: *receive active* (RxA) of valid input data to be consumed, and *receive free* (RxF) of free buffers ready to be re-used).

For a device using a request interface, there are *request active* (RqA) and *request free* (RqF) queues.

While Figure 4.3 shows a driver metadata region, the virtualiser metadata region has the same structure, the only difference being that data region references use offsets from the start of the data area, instead of I/O addresses.

```
1  #define QUEUE_SIZE (1<<QUEUE_LOG_SIZE)
2  struct buffer_descr {
3      uint64_t io_or_offset;
4      uint16_t length;
5  }
6  struct queue {
7      uint16_t head;
8      uint16_t tail;
9      uint32_t consumer_signalled;
10     struct buffer_descr buffers[QUEUE_SIZE];
11 }
```

Listing 4.1: Control region queue data structures.

Listing 4.1 shows the queue data structures. Each queue has a separate head and tail index, queue entries between those indices are valid in that they contain an offset to a buffer in the respective data area.

Each queue has exactly one *producer* and one *consumer*. Only the producer updates the tail, and only the consumer updates the head.

Tracing the path of a Tx buffer from virtualiser to driver, the virtualiser (producer) hands *ownership* of a DMA buffer to the driver (consumer) by inserting the buffer into the *active* queue (changing its state from ① to ②), from where the driver collects it (changing state from ② to ③). When done with a buffer, the driver (who for this queue is the producer) hands it back to the virtualiser (now the consumer) by inserting it into the *free* queue (③ to ④), from where the virtualiser can collect it (④ to ①).

Specifically, as shown in Listing 4.2, the producer enqueues a data buffer at the tail by checking there is at least one unused entry, inserting the new buffer there, and incrementing the tail pointer. Just before updating the tail pointer, the producer issues a *memory write barrier* to ensure that no writes are re-ordered by the compiler or the processor across this point. Dequeueing data buffers from the head is analogous, as per Listing 4.2.

Inserting a reference to a buffer into one of the queues changes ownership (see the definition of buffer states in Section 4.2.2). The inserting component loses the right to access the buffer, *as well as the descriptor referencing it*. The precise point of ownership transfer is Line 11 in Listing 4.2: Incrementing the queue's tail pointer by the producer makes the entry visible to the queue's consumer, which has then acquired ownership.

```

1 bool empty (struct queue *queue) {
2     return !((queue->tail - queue->head) % QUEUE_SIZE);
3 }
4 bool full (struct queue *queue) {
5     return !((queue->tail + 1 - queue->head) % QUEUE_SIZE);
6 }
7 int enqueue(struct queue *queue, struct buffer_descr buffer) {
8     if (full(queue)) return -1;
9     queue->buffers[queue->tail] = buffer;
10    memory_release();
11    queue->tail++;
12    return 0;
13 }
14 int dequeue(struct queue *queue, struct buffer_descr *buffer) {
15     if (empty(queue)) return -1;
16     *buffer = queue->buffers[queue->head];
17    memory_release();
18    queue->head++;
19    return 0;
20 }

```

Listing 4.2: Control-region queue management.

We can observe that ownership of data buffers is *defined* by where they are referenced: ownership of a queue entry implies ownership of the buffer it references. An interesting consequence is that for an active Tx buffer, ownership passes from the client to the virtualiser, from there to the driver and from there to the device. So, while the driver cannot access the buffer (it is not mapped in the driver's address space) it still *owns* it, for the sole purpose of being able to pass it to the device. Hence, ownership is a necessary but not sufficient condition for being able to access data.

A consequence is that a buffer can at any time be referenced by at most one queue – this is a core integrity condition of the scheme.

Lock-free updates to these data structures are possible by using the processor's property that *reads and writes of small integers are atomic*. The obvious data race between consumer and producer is benign. The memory barrier is sufficient to ensure consistency.

4.2.4 Device metadata region

The *device metadata region* plays a similar role in the driver-device interface as the *driver metadata region* does in the virtualiser-driver interface.

A core difference between the two regions is that while the *driver metadata region* is a standard shared-memory region (shared between two processes running on the CPU), the *device metadata region* is a DMA region (shared between the driver process on the CPU and the device hardware).

The main practical difference is that DMA bypasses the cache. On architectures that do not guarantee consistency between DMA and caches (anything but Intel), this requires explicit

steps to ensure consistency. We prescribe that on such architectures *the device metadata region is mapped uncached in the driver's address space*.

```

1  #define HW_QUEUE_SIZE (1<<HW_QUEUE_LOG_SIZE)
2  struct HW_buf_descr {
3      uintptr_t io_address;
4      size_t    length;
5      status_t  status;
6  }
7  struct HW_queue {
8      uint32_t head;
9      uint32_t tail;
10     struct HW_buf_descr buffers[HW_QUEUE_SIZE];
11     struct buffer_descr buffer_desc[HW_QUEUE_SIZE];
12 }

```

Listing 4.3: Typical network device queue data structures.

As it interfaces with the device hardware, the *device metadata region's* data structures are defined by the device interface protocol (see Section 3.2). Network devices (NICs) generally use similar queue structures as what we specified for the *driver metadata region* in Section 4.2.3 although there is usually one single queue each for transmitting and receiving; we will call those HW_Tx and HW_Rx, respectively. Listing 4.3 shows a representative example, which we will assume for the following discussion, noting that details may differ between devices.

Instead of separate *active* and *free* queues, hardware (HW) uses the status field of each queue entry. On output the device processes only entries marked as ready, and, once processed, sets the status to free or an error condition. On input, the device uses entries marked as free and, once data is delivered, changes the status to ready (or error). Note that the device only knows about the location of the memory area that contains the queue; this is provided to the device via a control register. The head and tail pointers are pure software constructs.

Listing 4.4 shows how the driver manages those queues. For simplicity, this pseudocode assumes that the hardware processes Tx buffers in queue order (which may not be the case in reality).

4.3 Component models

4.3.1 Driver

The driver is event-driven, in line with conclusions from our earlier work [Ryzhyk et al., 2009]: It acts in response to

- transmit requests from the virtualiser (*transmit active*, T_a);
- data requests from the virtualiser (*data requested*, R_q);
- receive-ready notifications from the virtualiser (*receive buffers free*, R_f);
- data-available interrupts from the device (*receive available*, R_a);

```

1  bool HW_full (struct HW_queue *queue) {
2      return (queue->head - queue->tail + 1) % HW_QUEUE_SIZE == 0;
3  }
4  int HW_enqueue (struct HW_queue *queue, struct HW_buf_descr HW_buf_descr) {
5      if (HW_full(queue)) return -1;
6      queue->buffers[queue->tail] = HW_buf_descr;
7      memory_release();
8      queue->tail = (queue->tail + 1) % HW_QUEUE_SIZE;
9      return 0;
10 }
11 int HW_dequeue (struct HW_queue *queue, struct HW_buf_descr *HW_buf_descr) {
12     if (queue->buffers[queue->head].status == READY) return -1;
13     *HW_buf_descr = queue->buffers[queue->head];
14     memory_release();
15     queue->head = (queue->head + 1) % HW_QUEUE_SIZE;
16     return 0;
17 }

```

Listing 4.4: Device queue management (simplified).

- completion interrupts from the device (*transmit completed*, T_C).

Notwithstanding some differences in terminology, this model is a refinement of earlier work, where we argued for *active* device drivers [Ryzhyk et al., 2010] (in the sense of having their own thread of execution): The driver operates single-threaded in its own address space, handling requests from either the OS (via the virtualiser) or device interface. This model has the benefits highlighted by Ryzhyk et al. [2010]: The driver is free from most concurrency issues and does not need to concern itself with client state that is not explicit in the request to be processed.

The driver thread runs at higher *priority* than the virtualiser to ensure timely handling of interrupts as well as immediate response to virtualiser requests. Flow control (described below) prevents the driver from monopolising the processor in the case of high incoming network load. In the storage case, all reads are triggered (and buffers provided) by the client (via the virtualiser) so there is no need for flow control.

4.3.2 Virtualiser

The virtualiser (Virt) is responsible for presenting a physical device as a virtual device to one or more clients. Each control path (Rx/Tx/Rq) has its own virtualiser.

Virtualisation includes translating between (Client) virtual addresses and I/O space addresses seen by the device. I/O space addresses may be physical addresses, or may be translated by an IOMMU. The translation also includes a change of representation: the client-side interface of the virtualiser uses offsets relative to the start of a memory region as memory references, while the driver side uses actual addresses.

If an IOMMU is used, I/O-space mappings may be static (defined at build time) or dynamic (e.g. to dynamically make client-provided I/O buffers available to the device). If dynamic IOMMU mappings are used, managing the IOMMU is the responsibility of the virtualisers.

Static mappings generally imply the need for Copier components to keep client address spaces separated, and also imply trusting the Driver as well as the device hardware.

Virtualisers generally require access to the *data region*, so it is mapped in their address spaces. There are two reasons:

1. an Rx Virt needs access to packet headers to determine the destination (i.e. the client which should receive the data);
2. On architectures that do not guarantee coherence between caches and DMA accesses, the virtualisers need to do cache management on the data buffers, which (on most architectures) requires the buffers to be mapped in its address space.

On architectures that require cache management, this must happen *whenever a buffer is handed to the device* (via the driver). Specifically, the virtualiser must *invalidate* input (i.e. *free*) buffers before handing them to the driver, and must *flush* output (i.e. *active*) buffers before handing them to the driver. Speculative reads on input buffers may happen while the I/O is in progress, requiring the virtualiser to *invalidate* input (i.e. *active*) buffers again when receiving them from the device [Rutland, 2016, p. 33].

A virtualiser that has multiple clients presents each client with the illusion of exclusively owning the device. This requires partitioning (for storage) or multiplexing (for network-like devices). Details are device-specific and will generally be based on a policy.

For network devices, the Rx Virt generally does not require a specific run-time policy, other than the use-case independent policy of dropping incoming packets if the RxA queue is full. (But note that the size of the Rx queues in the virtualiser metadata region limits the number of Rx buffers a client can have, and thus their size enforces a policy on the amount of Rx traffic processed for the particular client.)

A network Tx Virt imposes some form of traffic-shaping policy, such as priority, round-robin or bandwidth limits. Furthermore, in the Tx path, an I/O buffer is permanently associated with a particular client: once the data is transmitted, the virtualiser will return the buffer to the client that had originally supplied it.

- *TxA not empty* notifications (i.e. data available) from the client;
- *TxF not empty* notifications (i.e. returning free buffers) from the driver.

The Tx Virt needs to transmit requests as quickly as possible, both to provide the best service to the client as well as to return free buffers. It therefore should run at a priority higher than the clients. Flow control prevents the virtualiser (as well as the driver) from monopolising the CPU.

Further optimisations of this signalling protocol should be possible and will be explored in the future.

4.3.3 Observations

As mentioned earlier, IOMMU management and cache maintenance is no concern of the driver, it is the responsibility of the virtualiser. Other than that, a virtualiser does little more than moving buffer references between queues. A Tx or Rq Virt will generally implement some traffic-shaping policy (while an Rx Virt is usually policy-free). In order to keep the implementation simple, this policy should be tailored to the specific use case. If a different

policy is needed, the Virt should be replaced with one implementing the new policy, rather than making it more complex.

Combining the discussion of priorities in Section 4.3.1 and Section 4.3.2, we have a priority assignment of $P_C < P_V < P_D$. This configuration is consistent with monolithic systems, where the OS generally runs at higher (effective) priority than apps. Also mentioned earlier, what we refer to as the “client” may in fact be a pipeline of components, possibly comprising copiers and a server providing an OS abstraction. The principle of priorities increasing along the pipeline from client to driver should also hold if the client is in fact a pipeline of components.¹

Priorities may not mean much in a multicore system, where each pair of communicating components may be located on different cores. Hence, as in any parallel system, components *must not make assumptions on relative execution order!* Nevertheless, to maximise location transparency, the proposed priority assignment should be maintained even in a multicore scenario. This will be essential if core assignments change dynamically, e.g. if an energy-management policy consolidates components in order to off-line cores.

4.4 Synchronisation

There are two ways to implement this model: as *active* or as *passive driver threads* (Figure 4.4). Each has some advantages and it is a-priori not clear which one is better. We need a thorough evaluation to settle on the preferred implementation.

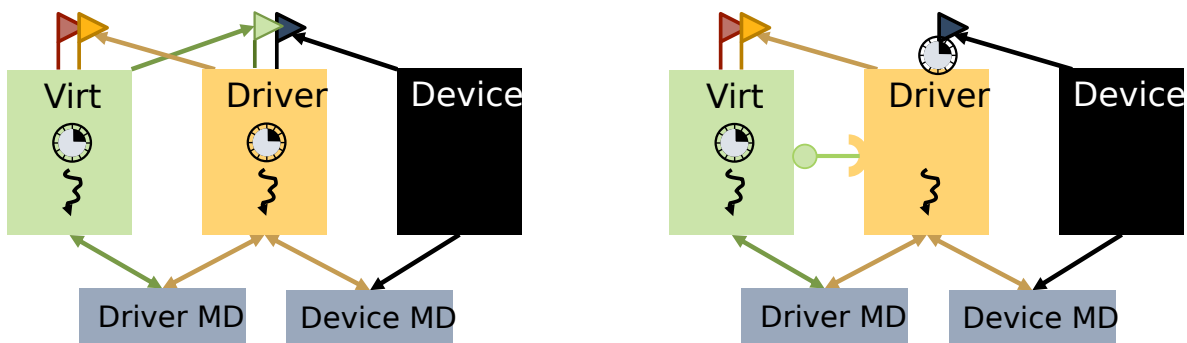


Figure 4.4: Active (left) vs passive (right) driver-thread model.

4.4.1 Active driver-thread model

In this model, the driver thread is *active* in seL4 MCS terminology, meaning it has its own *scheduling context* (SC). All its interfaces are semaphores represented by seL4 *notifications* (besides the shared memory *metadata regions*).

More specifically, the driver and virtualiser each have a *notification*. The virtualiser holds a badged *send* capability for the driver’s *notification*, the badge identifies the virtualiser to the driver. The IRQ is represented by a different badge representing the device.

The virtualiser performs its T_a , R_q and R_f operations by enqueueing descriptors in the *driver metadata region* and signalling the driver’s *notification*. The driver uses the badge to distinguish signals from the virtualiser and the device.

¹This priority assignment is the opposite of the original device driver framework and addresses some of the latter’s performance problems.

Similarly, the virtualiser has a *notification* for which the driver has a badged capability (and the virtualiser's client holds a differently badged capability to the same *notification*). The driver uses this *notification*, together with a status indicator in the HW queue, to perform the R_a and T_c operations.

Remember that the driver runs at higher priority than the virtualiser. Therefore, if both components run on the same core, the signals from the virtualiser to the driver are effectively synchronous, resulting in an immediate context switch to the driver (unless the driver is out of budget).

Listing 4.5 shows *simplified* pseudocode for an Ethernet driver. A more optimised version will, after processing notifications from the virtualiser, also check for pending IRQs from the device (which may have arrived while the driver was running on behalf of the virtualisers) and process these. Similarly, in a multicore system, virtualiser notifications may have arrived while processing IRQs and should be checked prior to returning. The driver will also, upon processing the Tx-completion IRQ, check whether there is work in the TxA queue.

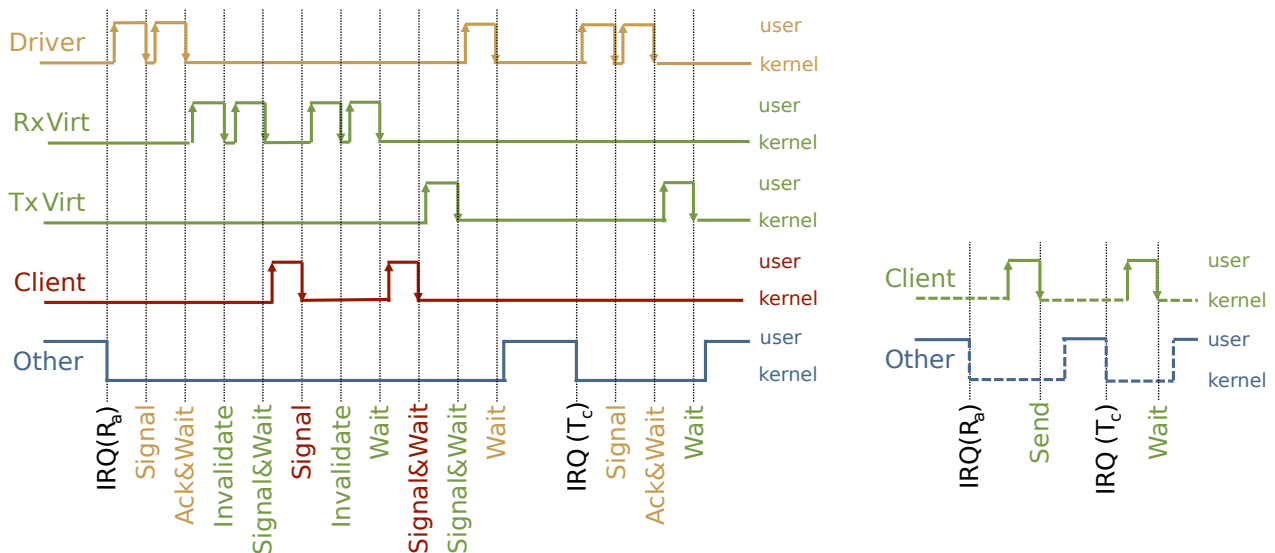


Figure 4.5: Mode switches for the active driver-thread model (left) compared to Linux (right) for a packet round trip in a single-core setup. For horizontal lines, solid indicates execution, dashed indicates suspension (blocked or preempted). For vertical lines, arrows indicate synchronous mode switches (system calls and returns) while dashed lines indicate asynchronous switches (interrupts or scheduling). “Other” refers to a lower-priority background activity.

Figure 4.5 shows the mode switches for this model compared to Linux. The diagram shows a total of 15 kernel entries for a complete packet round trip. However, if the system is under high load, it is less likely that all signals will be required by the driver, as it is more likely that the virtualiser is already awake and processing these queues, which is indicated to the Driver by a flag in the respective queues.

In contrast, the Linux system only has 4 kernel entries. The difference represents the inherent overhead of the microkernel-based design: On Linux, each client operation is a single system call (total 2), while in seL4 this corresponds to context switches to the virtualisers and the driver and back, significantly increasing the number of system calls. Furthermore, the usermode seL4 driver requires system calls to acknowledge IRQs. In addition, the cache management done by the virtualiser also requires system calls, as these operations are privileged on the Arm architecture (on RISC-V they can be delegated to usermode code,

```

1  handle_irq() {
2      while (event = clear_hw_events()) {
3          if (event & Tc) {
4              enqueued = false;
5              while (!full(\gls{tx}F) && buf = HW_dequeue(HW_\gls{tx})) {
6                  enqueue(buf, \gls{tx}F);
7                  enqueued = true;
8              }
9              if (enqueued && require_signal(\gls{tx}F)) signal |= \gls{tx}F;
10         }
11         if (event & Ra) {
12             enqueued = false;
13             while (!full(\gls{rx}A) && buf = HW_dequeue(HW_\gls{rx})) {
14                 enqueue(buf, \gls{rx}A); /* process input */
15                 enqueued = true;
16             }
17             if (enqueued && require_signal(\gls{rx}A)) signal |= \gls{rx}A;
18             while (!full(HW_\gls{rx}) && buf=dequeue(\gls{rx}F)) {
19                 HW_enqueue(buf, HW_\gls{rx}); /* return free \gls{rx} buffers */
20             }
21         }
22         if (event & error) {
23             fail;
24         }
25     }
26 }
27 main() {
28     initialise();
29     signal = init_done;
30     while (true) {
31         event = signal_and_wait(signal);
32         if (event & IRQ) {
33             hand_irq();
34             signal |= ack;
35         }
36         if (event & Ta) {
37             while (!full(HW_\gls{tx}) && buf=dequeue(TxA)) {
38                 HW_enqueue(buf, HW_\gls{tx});
39             }
40         }
41         if (event & Rf) {
42             while (!full(HW_\gls{rx}) && buf=dequeue(RxF)) {
43                 HW_enqueue(buf, HW_\gls{rx}); /* return free Rxbuffers */
44             }
45         }
46     }
47 }

```

Listing 4.5: Ethernet driver pseudocode.

reducing the required system calls). Linux's synchronous I/O API also avoids one kernel entry by forcing the client to block until the send is completed, while the sDDF API is asynchronous (to enable execution concurrent to I/O without forcing a multi-threaded design).

Note that interrupt batching, which occurs automatically if further interrupts arrive while the driver is processing packets with IRQs disabled, will on average reduce the number of kernel entries per round-trip by two for the seL4 system and by one for Linux.

Flow control is achieved by three means:

1. limiting the size of the receive queue in the *device metadata region* (which rate-limits interrupts by forcing the device to drop packets and thus limiting the input packet rate);
2. limiting the size of the send queue in the *driver metadata region* (which rate-limits the virtualiser's output requests) – this is unlikely to achieve much, given that the driver runs at higher priority than the virtualiser;
3. limiting the driver thread's budget (which limits the amount of work it can do in a period, irrespective whether the work is on behalf of the client or to service IRQs).

For a NIC, the *period* of the driver's SC should be the (desired) minimal inter-arrival rate of packets. The *budget* is part of flow control. Its choice depends on a number of factors, in particular the relative speeds of CPU and NIC: If the CPU is fast enough to process packets at line speed, the budget should be large enough to process one incoming and one outgoing packet. If the CPU is not fast enough, the budget may need to be reduced to allow the back-end to keep up with I/O.

If the driver's budget is depleted, the driver is suspended by the kernel until its budget is replenished at the end of the period. This suspends processing IRQs as well as virtualiser requests.

Note that on each invocation, whether on an IRQ or a client request, the driver needs to check for further events that may have happened while it was processing the current event: IRQs may arrive at any time and the driver must poll for pending interrupts before returning (resulting in batching). Similarly, more client requests may arrive during driver execution, if the driver's budget was exhausted during the processing of an event or the virtualiser runs on a different core than the driver.

More batching can be forced by using the NIC's *IRQ coalescing* feature, i.e. deferring the T_c interrupt for some amount of time or until a certain number of transmits have been completed. This is unlikely to provide significant benefit, at least for the 1 Gb/s (or less) NICs typically used in embedded systems. It might be useful for faster NICs.

4.4.2 Passive driver-thread model

In this model, the driver thread is *passive* in seL4 MCS terminology, meaning it does not have its own SC and can only run on a borrowed SC. As it runs at higher priority than the virtualiser, this makes the client→virtualiser invocation synchronous if both components share a core. This naturally leads to the virtualiser-side interface being an *endpoint*, which is invoked by the virtualiser as a *protected procedure call*, passing the virtualiser's SC along for the driver to execute. Thus, T_a , R_q and R_f are invocations of the driver's *endpoint* (with the opcode passed as an argument), while R_a and T_c are returns from the *endpoint* invocation.

As the driver does not have its own SC, its *notification* (which delivers the IRQ from the device) must be active, i.e. have an SC that is lent to the driver's thread.

The number of mode switches in this model is similar to the active driver model (possibly one higher). There is a performance advantage, as a protected procedure call to a passive PD is faster than signalling a notification to a higher-priority PD. A potential drawback is that this forces the driver and virtualiser to be co-located on the same core (and the virtualiser with its client, if it is also passive). The performance trade-offs are not obvious and need evaluation.

Flow control is achieved by three means:

1. limiting the size of the receive queue in the *metadata region*;
2. limiting the size of the send queue in the *control region* (same caveat applies as for the active driver thread);
3. limiting the budget of the driver's *notification* (which implicitly limits the interrupt rate without limiting the driver's ability to respond to virtualiser requests, but see below for more detailed discussion).

For a NIC, the period of the *Notification's* SC should be the (desired) minimal inter-arrival rate of packets. The budget primarily needs to allow for IRQ handling, as requests from the virtualiser execute on the latter's budget. But note that, even if serving a virtualiser request, the driver needs to poll for IRQs before returning, so it may handle interrupts on the virtualiser's SC.

However, the virtualiser is blocked during that time, unless the driver has a timeout exception that allows it to hand control back to the client if the budget runs out. This would, however, complicate driver logic (including re-introducing a degree of concurrency). On the other hand, blocking the virtualiser while the driver is waiting for a budget replenishment reduces the virtualiser's ability to operate concurrently.

For the R_a IRQ, the budget must be sufficient to enqueue the packet(s), notify the virtualiser, and acknowledge the IRQ. Similar, the T_c IRQ needs to have sufficient budget to update queues, notify the virtualiser and acknowledge the IRQ.

Note that IRQ coalescing leads to multiple packets processed for a single R_a interrupt, while IRQ masking leads to multiple buffers freed for a single T_c interrupt. The budget must allow for that.

4.4.3 Two-component driver

It is possible to split the driver into two separate components, one interfacing to the virtualisers (i.e. handling T_a , R_q and R_f events) and one interfacing to the device (handling IRQs). This allows serving both types of requests concurrently by running the two PDs on different cores (and *only* makes sense for this case).

The idea of a two-component driver may look like heresy in light of the discussions of Section 4.3.1, specifically the stated goal of avoiding error-prone concurrency control inside the driver. However, it is actually a straightforward extension and does *not* require extra concurrency control inside the driver. The two driver components do not access the same software-provided data structures. Specifically, only the virtualiser-interfacing component accesses the T_xA queue of Figure 4.3, while only the device-interfacing component accesses the T_xF and R_xA queues.

The only potentially competing access to a virtualiser-side queue would be for returning free buffers from RxF to the hardware queue. In order to avoid explicit concurrency control and maintain the single-producer, single-consumer property of the queues, we need to restrict accessing this queue to the virtualiser-side component (Line 43 of Listing 4.5) – the identical code in the IRQ handler (Line 19 of Listing 4.5) is a performance optimisation, not a functional requirement.

The performance benefit of this optimisation in the multi-component case is not obvious without detailed evaluation. An evaluation performed by Parker [2023, Sect. 7.4.3] showed no performance benefit (in fact, a slight performance degradation) and we therefore do not investigate this model further for now.

4.4.4 Time-triggered architectures

The model readily adapts to *synchronous* (aka. *time triggered*) architectures [Kopetz, 2003], where each component executes at a pre-determined time point, irrespective of interrupts. In this case, none of the components signal notifications, synchronisation is exclusively via timer signals. If a component receives a signal, it processes its queues and then waits for the next signal.

4.4.5 Discussion

The design currently has two major open questions that need more analysis.

Research Question 1.: *Active or passive driver threads?* Considerations:

- C1.1:** The active *notification* for receiving IRQs arguably allows the budget to be adjusted to the driver's needs, but the need for checking for new work before returning reduces the benefit.
- C1.2:** An *endpoint* results in the virtualiser being charged for the time the driver consumes on its behalf. However, this benefit is limited, as the IRQ processing is still not accounted to a specific client.
- C1.3:** Invoking a passive thread through an *endpoint* avoids switching the scheduling context, while signalling a higher-priority thread forces a switch of SC, incurring higher cost. However, the active-thread design comes at the cost of one extra system call.
- C1.4:** If the passive driver thread is blocked on budget replenishment, it blocks the virtualiser, reducing the benefits of using the budget for flow control.
- C1.5:** On multicore, the *notification* forces the driver to run on a specific core, while the *endpoint* forces it to run on the virtualiser's core. Which model is better likely depends on the application scenario.
- C1.6:** The virtualiser may want to use a watchdog to prevent indefinitely blocking on an untrusted driver. As long as the virtualiser only synchronises with *notifications*, the watchdog timeout can be signalled to the virtualiser's *notification*, terminating the wait. If the virtualiser invokes the driver by IPC, then the watchdog would have to explicitly reset the virtualiser to abort the IPC, a somewhat more complicated operation.²

²The simpler aborting logic could be extended to IPC via a kernel change recently discussed: allowing specified signals to abort an IPC. The full implications of this change are not yet fully understood, so this change may or may not happen.

Similar questions arise for virtualiser threads, which may also be active or passive.

Research Question 2.: *What are appropriate budgets for the driver (and virtualiser), and how are they determined?* Considerations:

- C2.1:** How important is the driver / IRQ *notification* budget for rate-limiting the driver? Is limiting buffer size sufficient, in which case the driver could simply be given a full budget, removing the issue of the suspended (passive) driver blocking the virtualiser?
- C2.2:** A virtualiser probably still only has small budgets, and a period matching that of the driver.

Chapter 5

Device Classes

Having described the general model of device drivers and their interfaces, we will now refine those to specific device classes.

5.1 Network

5.1.1 Main properties

Network devices are characterised by output being client-initiated while input is spontaneous. While in practice inputs may result from some client action (e.g. a request to a web or file server), this is the result of higher level protocols that are invisible to the driver (and device).

As such, network drivers do not have a request interface, only a transmit and a receive interface. A typical network structure is shown in Figure 5.1.

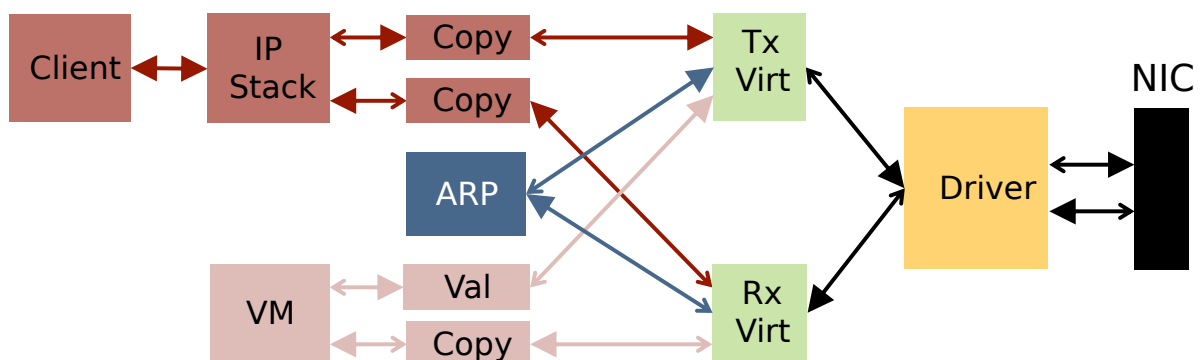


Figure 5.1: Modularised design of a networking architecture, showing copier (Copy) and Validator (Val) components for isolating untrusted clients, as well as an ARP component that handles broadcast requests.

Network devices have been our running example throughout Chapter 3 and Chapter 4, so much relevant information has already been covered. Here we summarise the main characteristics:

- The Tx and Rx paths are separated, each with its own virtualiser, data region and a metadata region between any two components, as shown in Figure 4.2.

- Each of these interfaces consists of two queues. The *active* queue (TxA/RxA) points to valid data buffers which the producer (Driver on input, Client on output) hands to the consumer. The *free* queue (TxF/RxF) points to buffers that are returned by the consumer to the producer for re-use, see Figure 4.3.
- Each I/O buffer has at any time a defined *owner*, and is in one of the four states introduced in Section 4.2.2:
 1. source-owned and in use
 2. destination-owned and idle
 3. destination-owned and in use
 4. source-owned and idle.
- The states are implicitly encoded in where the buffer is referenced, see Figure 4.3. Consequently, buffers change state and owners by being moved on and off the queues by either the producer or the consumer, see Section 4.2.3 for details.
- The driver's Rx and Tx interfaces are entirely separate, in that the driver will only move buffers between queues of the same interface, not across interfaces.

5.1.2 Ethernet

Ethernet represents an important sub-class of network devices. It represents a well standardised category, with common properties such as frame size. Specifically, Ethernet devices have a standard transfer granularity, called a *frame*, of 1.5 kB. Many Ethernet devices also support larger frames, called *jumbo frames*, but we do not consider them here.

Server platforms frequently feature *self-virtualising NICs*, with multiple interfaces to the same underlying device. These enable secure pass-through access of the same device to multiple virtual machines. Using this feature generally involves extra work by the hypervisor to handle broadcast traffic. For now we do not plan to support this feature.

5.1.3 Status

The specifications for Ethernet are tested and evaluated, they are **stable**.

5.2 Asynchronous serial busses

5.2.1 UART

UART (Universal Asynchronous Receiver-Transmitter) I/O is an asynchronous serial communication standard used on most consumer electronics for bidirectional text communication.

UART is very similar to networking, in that output is client-initiated, and input is spontaneous. At the hardware level, communicating terminals need to agree on bit-rate, number of bits per character, and whether a parity bit is included.

UART tends to be much lower bandwidth than networking, with most serial ports having a maximum throughput around 4Mb/s. Consequently, the UART subsystem merges the data being transferred into the same metadata region as the queue head and tail.

The UART subsystem is referred to as “serial” within the sDDF.

Note: More **details to come**.

5.2.2 USB

Details to come.

5.3 Synchronous serial busses

5.3.1 I2C host

I2C (inter-integrated-circuit) is a half-duplex two-wire synchronous serial bus used for interfacing between chips. The bus architecture consists of some number of controllers which control targets (peripherals). The sDDF supplies a driver for hardware I2C controllers with 7-bit addressing.

All targets have a bus address which is typically 7 bits. A controller can start a transaction by sending a start bit followed by an address. The final bit of the address specifies if the operation is a read.

On every rising edge of the clock after the start and address, the controller will write a bit or read a bit until there is no more data available. The controller sends a STOP condition to indicate it is done writing, or the target send a NACK to indicate no more data can be read. Only one target can ever be addressed at any one time and all interactions must be instigated by a controller.

An important convention for I2C is the usage of device register addressing. This is not a part of the standard but is used in most peripherals. Registers are addressed by performing a 1-byte write operation to send the target register to the device before continuing to perform the expected operation. For example, a read to register 0x55 begins as:

- Start condition,
- address of peripheral with read bit set to zero (write),
- write register address (0x55),
- (repeated) start condition,
- address of peripheral with read bit set.

For writes, the first byte of the written data is interpreted as the register address. We refer to the “read from register” operation as a “write read” within our materials for brevity.

Transport layer

The I2C subsystem has each client’s data region mapped directly into the driver’s address space instead of using DMA buffers to share information as we assume a non-DMA compatible host controller.

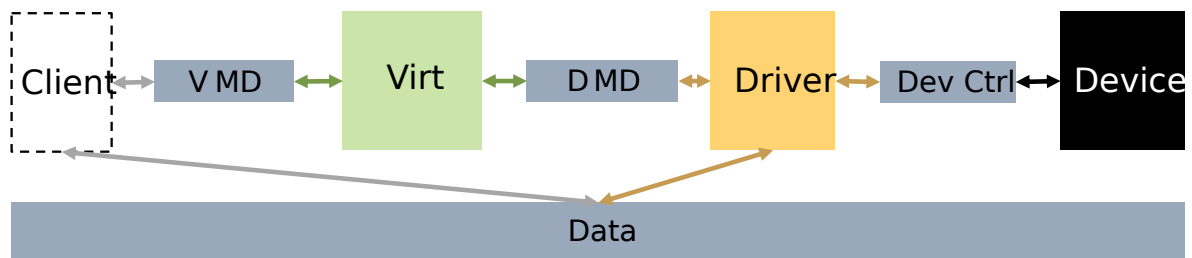


Figure 5.2: Memory regions (grey) shared between software components and the device. “MD” represents metadata regions of the driver (D) and virtualisers (V).

As a protocol with a controller-target architecture, the I2C protocol has an asymmetrical queueing design because there is very little data in responses relative to requests. There is no notion of bidirectional communication, clients instead request reads and writes and receive only simple acknowledgements in return. This is contrasted with the identical RX and TX queues in bidirectional (symmetric) driver subsystems.

Each PD in the driver subsystem is connected using a request and response queue, where requests contain specifications for I2C transactions and responses contain a simple error state and number of bytes written or read.

Requests are structured as a series of commands, where each command corresponds to a single read or write operation against a single buffer in the data region. Each buffer is an arbitrary pointer to the data region as specified by the client.

The request queue has a special behaviour: the tail pointer (most recently enqueued member) is not updated immediately upon enqueue. Instead, a staged tail is maintained by the client and is “committed” once a client has inserted all the commands it needs in a transaction. This prevents race conditions in multicore environments by ensuring transactions are transmitted atomically.

The first command in a request is the header, specifying the bus address that all commands target and the number of commands in the request. Since I2C is a serial protocol which has all interactions specified by the controller, it is very important that transactions are transmitted in the order clients expect. It is impossible to address a different client in the midst of a transaction without breaking the ongoing transaction! The header is used to group commands to ensure monotonic, contiguous transmission.

Commands are implemented as a tagged union such that they can function as a header or regular command as shown in Listing 5.1. A flag mask sets properties of the command such as whether to insert a stop condition or if the command is a header.

It is an invariant that a header specifying a transaction of N commands must be followed by those N commands immediately. It is not possible to enqueue partial requests as this will significantly complicate the design of the device class.

The response queue is a simple first-in, first-out queue containing simple response structures. Each response contains the I2C bus address of the corresponding request, the error value and the number of bytes written or read.

```

1  union {
2      /* Location of read destination / write data in data region */
3      uint8_t *data;
4
5      /* Fields for header members */
6      struct {
7          uint16_t batch_len;
8          i2c_addr_t address;
9      } i2c_header;
10 } payload;
11 /* Size of read/write operation. Max 256 back-to-back reads / 64KiB write */
12 /* Unused if this is a header command */
13 uint8_t data_len;
14 i2c_cmd_flags_t flag_mask;
15 } i2c_cmd_t;

```

Listing 5.1: I2C command structure.

Clients and libi2c

Clients can request I2C transactions by composing a chain of requests locally and enqueueing them to the virtualiser queue. The virtualiser will process the contents of the queue upon being notified by the client. The client can manually compose an arbitrary transaction by creating commands or it can use libi2c to simplify transactions.

libi2c is a simple blocking interface to the I2C driver subsystem, supplying functions to compose single-command transactions. It also supplies non-blocking variations of all calls to enable usage of alternative concurrency models.

Whether the client used libi2c or not, the client will be signalled and a response struct will be returned by the virtualiser once the driver is finished working on the transaction.

Virtualiser

There is a single virtualiser in the I2C driver subsystem, connected to all clients with request and response queues and using notifications for IPC. The virtualiser is responsible for ensuring that incoming requests are valid and processed in the correct order. It also is responsible for translating data buffer addresses in commands to their mappings in the driver address space.

Unlike most other sDDF virtualisers, the I2C virtualiser must operate freely from PD notifications instead of being driven transiently by them. This is to say that rather than simply forwarding requests or responses as they arrive, the virtualiser instead builds an internal queue of work which it will process independently. The driver expects to only receive fully formed transactions in its queue and the virtualiser must thus always wait until enough room exists to forward work. This means that the virtualiser will try and forward requests both upon being notified by a client as well as when the driver notifies it to handle a response.

Incoming work from clients is enqueued in an internal queue structure and is guaranteed to be processed in order of arrival. This is a necessary policy to allow monotonic request processing without livelock.

Whenever the virtualiser is awoken, it will perform the following steps:

- If signalled by the driver, accept and forward all responses to clients. Signal clients who received a response.
- If there are any requests in the virtualiser internal work queue, attempt to process all of them. Any that cannot fit in the driver queue are left behind.
- If all deferred work was completed and a client signalled the virtualiser, handle the incoming work from that client. Process requests immediately if they fit in the driver queue and defer anything that remains. If previous deferred requests could not complete, the virtualiser adds the new request(s) to the back of the internal queue.

The addressing unit for the I2C device class is the bus address of peripherals. Only one client may have access to any particular bus address, and the bus address is used as a cookie to direct responses back to clients. Currently, clients can claim an address for their use via a protected procedure call to the virtualiser although this is planned to be replaced with build-time configuration in the future.

If any incoming request targets an address that doesn't belong to the requesting client, it will be returned as an error by the virtualiser.

Driver

The I2C driver is responsible for driving the I2C hardware to perform transactions.

Although I2C is a very simple protocol electrically, driving I2C hardware according to the contract specified by incoming commands requires a large amount of bookkeeping.

The I2C driver is implemented as a general-purpose finite state machine in order to promote reusability and robustness. A well-designed finite state machine eliminates most invalid software states and is a natural target for formal verification.

The state machine has six states, each representing a different phase of interaction with the request or response queues.

The states are:

- Idle. Go to sleep if the request queue is empty, otherwise go to Request state.
- Request. Find the next valid request and load it into driver data. Go to Select Command.
- Select Command. Choose a command to operate on from the current set and go to Command. If the request is done, go to Return.
- Command. Perform reads or writes using the hardware to complete the command. Once the the hardware needs to perform work, go to sleep and set Command Return as next state.
- Command Return. Extract bytes read / error status from hardware and add to driver data. Go back to Command if more work remains for current command, otherwise go to Select Command.
- Return. Create response queue entry using bytes read and error status, enqueue and signal virtualiser. Go back to Idle always.

If any state encounters invalid data or the hardware reports an issue, the state function will set an error value and jump to the Return state.

5.3.2 SPI

Details to come.

5.4 Storage

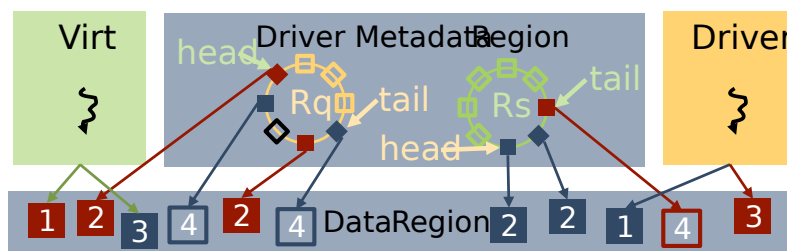


Figure 5.3: Storage device transport layer, showing the *control* and *data regions*. The numbers in the buffers in the data region indicate the buffer state, as defined in Section 4.2.2, where output buffers are red, input buffers blue. The black, empty node in the Rq queue indicates a *barrier* request.

5.4.1 Main properties

Storage devices deliver input upon specific requests rather than sporadically. As such they have *request* (Rq) and *response* (Rs) interfaces instead of the Tx and Rx interfaces that network devices use. Unlike network devices there is no separation into two control and data paths. Specifically:

- There is a *single data region*.
- There is a *single virtualiser*.
- There is a *single driver metadata region* that contains two queues: the Rq and Rs queues, as shown in Figure 5.3.
- The virtualiser uses the Rq queue to pass request descriptors to the Driver. Each descriptor holds either a **read**, a **write** or a **barrier** request, where read or write requests reference I/O buffers.
- The Driver dequeues a request from the Rq queue and (logically) hands it to the device.
- Buffer states are as defined in Section 4.2.2.
- Unlike for network devices, the queues reference a mixture of input and output buffers in arbitrary order. Inserting into the queue changes a buffer's state and owner. Removing from the queue changes the buffer's state but not the owner.
- Specifically, the Rq queue passes buffer ownership from the virtualiser to the device. Until a buffer is returned in the Rs queue, the virtualiser (or its client) must not access it. Buffers referenced by **write** requests will not be changed by the device. Buffers referenced by **read** requests will be filled by the device. Buffer state for *read* request

becomes *source-owned and idle* (no useful data in the buffer); for a *write* request it becomes *destination-owned and idle* (contains valid data).

- Likewise, the Rs queue passes buffer ownership from the device to the virtualiser (and on to the client). Inserting a response into the Rs queue indicates that the device will not access the buffer referenced in the response. The buffer state is *destination-owned and idle* for both read and write requests.
- As the device may reorder requests, the Driver may return buffers to the virtualiser (via the Rs queue) in an order different from the order they were inserted into in the Rq queue.
- The **barrier** request can be used to limit re-ordering: All requests before the barrier must be completed by the Driver/device before any of the requests after the barrier are initiated. The barrier request only ever exists in the Rq queue and references no buffer.

5.4.2 Details

Storage devices transfer data at the granularity of a *block*. In traditional rotating disks, this is the *sector*, typically 512 B (or a small multiple). These days, storage devices are large and a small amount of internal fragmentation is of no relevance. Furthermore, the setup time for a small I/O operation dominates the actual transfer time. As such, we see no benefit of supporting such small transfer sizes, and instead specify the size of transfers (and thus blocks) as a multiple of the platform's base page size (usually 4 KiB).

Where a device has an *optimal transfer size* that is larger than the page size, we use that as the block size.

Note we do not plan to support legacy devices such as CD-ROM.

Our model of a storage device is thus based on the following properties:

- The device has a *block size* that is a small integer, specifying the size in multiples of the base page size (usually the page size is 4 KiB and the block size is one).
- Large sequential (in DMA space) transfers tend to have a performance advantage, so the client (file server) should be able to issue requests larger than the block size.
- The device may support a scatterlist of DMA blocks for output and this may be faster than a sequence of individual output requests. Hence the client should be able to issue a batch of requests at once.
- The device can queue many read requests internally (typically a small power of two, $2^6 \dots 2^8$).
- Read requests can complete out-of-order (the device may re-order requests to increase spatial locality).
- The device may cache write requests and perform them out-of-order (necessitating the use of barriers to maintain write coherence).
- Many (but not all) storage devices are slow, so caching and asynchronous operation is important.
- Solid-state storage devices wear out (they typically have limited write cycles). The controller transparently re-maps bad blocks, maintaining the illusion of a contiguous I/O

space. It can be queried in order to estimate lifetime. Many storage devices use SMART [Wikipedia] for this. We plan to support a SMART interface in the future.

This leads to the set of request and response structures in Listing 5.2.

```
1  #define QUEUE_SIZE (1<<QUEUE_LOG_SIZE)
2  enum request_code {
3      READ_BLOCKS,
4      WRITE_BLOCKS,
5      BARRIER,
6      FLUSH
7  };
8  struct blk_request {
9      enum request_code storage_command;
10     uint32_t block_number;
11     uint16_t count;
12     void *address;
13     int32_t id;
14 };
15 struct blk_response {
16     enum {
17         SUCCESS,
18         /* various error codes */
19     } result;
20     uint16_t count;
21     uint16_t success_count;
22     void *address;
23     int32_t id;
24 };
25 struct blk_req_queue {
26     uint32_t head;
27     uint32_t tail;
28     bool     plugged;
29     struct request buffer[QUEUE_SIZE];
30 }
31 struct blk_response_queue {
32     uint32_t head;
33     uint32_t tail;
34     struct response buffer[QUEUE_SIZE];
35 }
```

Listing 5.2: Storage request and response queue data structures.

These are similar to the queues for the Ethernet devices. The main difference is the addition of a `plugged` boolean as an optimisation. When `plugged`, the driver is meant not to handle this or any subsequent requests. This allows the client (filesystem) to queue many requests, up to the size of the queue, before releasing the driver to queue them to the device. The device then has a queue of requests it can reorder for optimum device throughput (making use of a scatterlist if supported by the device). The client needs to notify the driver when it removes the plug, in the same way it notifies the driver when adding a request to an initially empty

request queue.

The `id` field is to aid in matching requests and responses; it is set by the client, maintained by the virtualiser, and copied into the response by the Driver. This field is not strictly needed, as the request and response can be matched on `address`, but may simplify the client logic.

The `success_count` field in the response structure indicates the number of blocks successfully transferred. The `status` indicates the status of the first failing transfer, or `SUCCESS` if all requested blocks were transferred correctly. Thus a request that asked for 128 blocks to be transferred, where there was a seek error on the 67th, would return `success_count=66`, and `status=SEEK_ERROR`.

In addition to the queues and shared-memory areas, there is a shared *information page* that contains the information a client might want about a storage device, and information for monitoring the device. The exact layout and content is still to be determined, for now we use the fields in Listing 5.3.

```
1 struct storage_info {
2     char serial_number[64];
3     bool read_only;
4     bool ready;
5     uint16_t blocksize;
6     uint16_t queue_depth;
7     uint16_t cylinders, heads, blocks; /* geometry to guide FS layout */
8     uint64_t size; /* number of blocksize units */
9 };
```

Listing 5.3: Storage information region data structure.

The only non-obvious field is `ready`: It is initially `FALSE`. The driver switches it to `TRUE` when the data in `struct storage_info` is valid, and the driver is ready to accept requests. This allows disk drives to spin up, and the driver to query information about the attached storage, before file systems can access the device.

The virtualiser presents a very similar interface. We have two virtualisers designed, and one implemented at present.

One virtualiser reads the block device's partition table, and presents each partition on the device to a separate client.

The other lives inside a virtualiser/driver VM, and presents a single file on the driver's filesystem as a virtual block device to a client.

5.4.3 Status

The basic design is **stable**, but details of specifications are **subject to change**.

5.5 Sound

5.5.1 Main properties

Sound devices expose a series of *streams* each of which either *play* or *record* (capture) audio. This audio is transmitted through buffers of pulse-code modulation (PCM) frames, where each frame contains a snapshot of the amplitude of each audio channel at a particular point in time.

The driver exposes information about each stream to clients through shared memory. For each stream, the driver specifies supported formats, rates, the stream direction (playback or capture), and how many channels the stream supports. The driver signals to the client that all streams are ready by setting the shared `ready` flag to true. Until then, clients must assume no streams are available (usually busy waiting).

Similar to storage drivers, sound drivers communicate with clients through two pairs of request / response queues: one for commands and one for PCM transfer. A stream has the following life cycle:

1. Take – take ownership of the stream, specifying the number of channels, format and rate.
2. Prepare – allocate resources for playback.
Client sends buffers to driver for pre-buffering.
3. Start – start playback or recording.
Driver begins responding to buffers.
4. Stop – stop playback or recording.
5. Release – free stream resources.

5.5.2 Message Protocol

The format of commands and PCM requests is shown in Listing 5.4. The type of a command is specified by its `code` field, and `stream_id` denotes the stream the command refers to. The field `cookie` is copied over to the corresponding response so the original message can be identified. Commands are sent in the `cmd_req` queue, and responses are received in the `cmd_res` queue. The field `set_params` contains stream parameters for a *take* request, and `status` is set for responses to signal the result of a request.

To both play and record PCM data, the client sends PCM request through the `pcm_req` queue. The fields `addr` and `len` refer to a shared PCM data buffer. On playback, this buffer will contain PCM frames to play and on recording this buffer will be filled by the device. Once the driver has finished with the buffer, it will respond on the `pcm_res` queue updating the `status` and `latency_bytes`. This process differs slightly for playback and recording.

The sound data region's ownership model is identical to Storage (Section 5.4.1), where playback is equivalent to writing and recording is equivalent to reading.

```

1  typedef enum {
2      SOUND_CMD_TAKE,
3      SOUND_CMD_PREPARE,
4      SOUND_CMD_RELEASE,
5      SOUND_CMD_START,
6      SOUND_CMD_STOP,
7  } sound_cmd_code_t;
8
9  typedef struct sound_pcm_set_params {
10     uint8_t channels;
11     uint8_t format; // SOUND_PCM_FMT_*
12     uint8_t rate;   // SOUND_PCM_RATE_*
13 } sound_pcm_set_params_t;
14
15 typedef struct sound_cmd {
16     sound_cmd_code_t code;
17     uint32_t cookie;
18     uint32_t stream_id;
19     union {
20         sound_pcm_set_params_t set_params; // Set on TAKE request
21         sound_status_t status; // Set on all responses
22     };
23 } sound_cmd_t;
24
25 typedef struct sound_pcm {
26     uint32_t cookie;
27     uint32_t stream_id;
28     uintptr_t io_or_offset;
29     unsigned int len;
30     // Only used in responses.
31     sound_status_t status; // SOUND_S_*
32     uint32_t latency_bytes; // play/record latency in bytes.
33 } sound_pcm_t;

```

Listing 5.4: Sound definitions

5.5.3 Playback

A minimal sound system consists of a *client*, *virtualiser*, and *driver* component, where the driver communicates to the *device* via DMA. To play audio, the client must first send *take* and *prepare* commands specifying the direction as `SOUND_D_OUTPUT` and setting stream parameters. Then, $n > 0$ PCM buffers must be pre-filled to `pcm_req` to fill the device's internal buffer, followed by a *start* command. Buffers are only responded to after they have been played. This means the i th pre-filled buffer will only be responded to after i buffers have been consumed. After the stream has been started, the client must **replay** (send again) these n pre-filled buffers. The i th replayed buffer will not be responded to until $n + i$ buffers have been played.

An example is shown in Figure 5.4 with $n = 5$ pre-filled buffers. The first three pre-fill buffers fill up the device's internal buffer, so buffers 4 & 5 are silently ignored. After *start* is sent, the

driver will initiate playback and will respond to the i th pre-filled buffer after i buffers have been played. While this is happening, the client *replays* buffers 1-5; buffers 1-3 are silently ignored and buffers 4-5 are sent to the device. There are several reasons for this pre-buffering behaviour.

- Pre-buffering reduces the latency of playback by starting the stream immediately as *start* is sent.
- The driver can choose to not support pre-buffering by simply ignoring all buffers sent before *start*, transparent to the client.
- This behaviour is required for compatibility with Linux's I/O virtualisation framework (VirtIO) sound implementation.

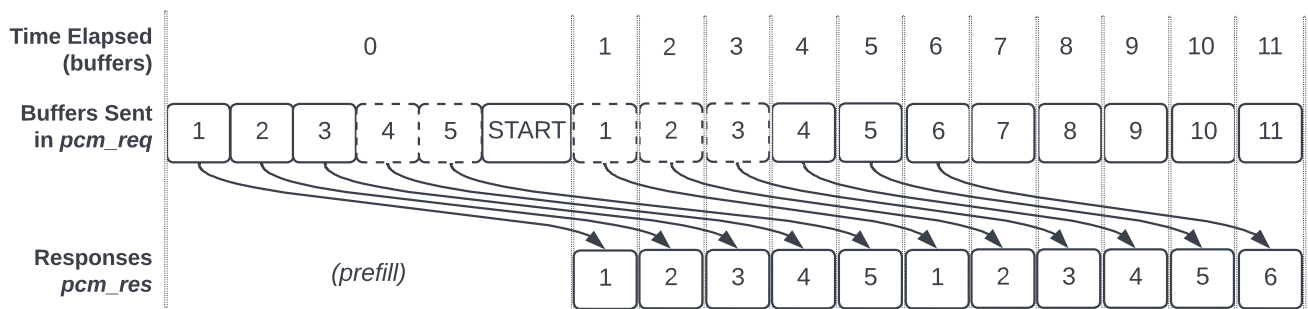


Figure 5.4: TX prebuffering

After all PCM buffers have been sent, the client sends a *stop* command followed by a *release* command to release stream resources.

5.5.4 Recording

Recording is almost identical to playback, however there is no “replay” behaviour. The client first takes and prepares the stream specifying `SOUND_D_INPUT`. The client then pre-sends empty buffers in `pcm_req` before starting the stream. On stream start, the driver will begin to respond to these with recorded PCM data. The client must continue to send empty buffers for the driver to fill. The client then sends a *stop* command and the driver fulfils the remaining requests.

5.5.5 Sound Virtualisers

The protocol is policy agnostic, however it has been designed to facilitate various virtualiser implementations. A simple virtualiser could use the *take* and *release* commands to allow clients to have temporary exclusive ownership over a stream. If another client tries to take a stream while it is in use, the request will fail. Alternatively, a virtualiser could allow multiple clients to access a stream at the same time by *mixing* (combining) the audio signals.

5.5.6 Status

This device-class specification is **subject to change**.

5.6 Display

Details to come.

5.7 Video Capture

Details to come.

5.8 Pin Control

5.8.1 Terminology

- **Port:** refers to an input or output line of a logic instance in the chip (e.g. UART, DDR, HDMI, I2C,...). For example, a UART instance have TX and RX ports. Not to be confused with pad.
- **Pad:** refers to the physical pin on the chip package that is connected to the outside world.
- **Client device:** a peripheral device on board that needs pin control configuration.

5.8.2 Main properties

As a chip contains a limited number of pads, it is not feasible to map each port to a separate pad. Hence, most pads are multiplexed between I/O *signals* (i.e. ports): A pad can be connected to one of multiple ports at a given point in time as appropriate for the intended use case. These signal-to-pin and pin-to-signal options are selected by the input-output multiplexer called a *pin controller* or *pinmux*. The pin controller is also used to configure other electronic characteristics of a pin, such as drive strength, bias, etc. All of these configurations can be programmed in software by writing to the pin controller's memory mapped registers.

The pin control driver as currently built relies on a prebuilt description that says what should be inserted into each pin control register. We have a python program that generates such a description from the device tree that describes the board and what is connected to it.

The pin control driver's code must be run once before any driver that uses the pins that it sets up.

A pin control driver is needed in two cases:

1. When some devices on the board are not initialised by the bootloader.
2. When there is a need for a device drivers to change pin control at run-time. For example, changing SD Card speeds on the Odroid-C4 involves switching a pin between a clock output and a GPIO input.

5.8.3 Message Protocol

Currently, our pin control driver pipeline only supports setting the default configuration. Dynamic reconfiguration of the pin controller is in progress. As such, the pin control driver has no software interface at the moment. **Details to come.**

5.8.4 Status

This device-class specification is **subject to change**.

5.9 Sensors, PWM, GPIO, etc

These device classes are all very similar, and tend to be low bandwidth — transferring one bit to a few bytes at a time. They fall into the “simple device” category introduced in Section 4.1; rather than using queues, the driver interfaces use PPC to directly pass data.

Pulse-width-modulation (PWM) is an output-only relatively high-frequency signal that, if averaged, yields a low frequency (or DC) signal. To program, one specifies a *period*, and a *pulse width* within that period — the *duty-cycle*. By varying the pulse width between 0 and 100%, the average voltage at the output varies between 0V and whatever the level ‘1’ voltage is (typically 1.8 V or 3.3 V for today’s SoCs).

General Purpose Input-Output (GPIO) pins can be programmed to act as input or output. As an output, such a pin can be set to a low or a high value; as an input it can detect whether it is connected to a low or high voltage level. On most SoCs, an input GPIO can also be programmed to generate an interrupt under various conditions. The conditions for these interrupts, and which GPIOs can generate them, are SoC-specific.

There are other devices that have similar characteristics.

5.9.1 Sensors

Sensors — things like tachometers for fans, temperature sensors, voltage and current measurement etc., — are generally read only; some have a low repeat rate (you need to leave a specified minimum time between readings).

Some sensors can be given set points to generate an interrupt when a measurement crosses a threshold.

Some devices have more than one sensor controlled from the same set of registers.

The interface for sensors therefore uses PPC, and has the following procedures that can be called:

get_number_sensors() returns the number of sensors controlled by this device.

get_reading(*n*) gets the current value measured by sensor *n*. If this function is called before a new reading can be obtained, it is implementation defined whether it returns the previous value, or an error.

set_threshold(*n*, *value*, *direction*) *direction* is one of three symbols: NONE, UP or DOWN. When the sensor reading exceeds *value* in that direction, a notification is generated to the caller. This procedure can fail if the driver cannot set interrupts for the threshold. If a threshold is already set, this call supersedes it. Calling with a direction of ‘NONE’ disables the notification.

5.9.2 GPIOs

A General-Purpose I/O (GPIO) pin can be set to be input or output; some GPIOs can trigger an interrupt if they are an input. In addition some platforms allow a GPIO to be set into a high-impedance output state, or an open-drain output state.

The sDDF uses PPCs to configure, set, and query GPIO state. GPIOs are set up at build time so that each PPC channel corresponds to a single pin. Thus the driver knows which GPIO is being used by the channel the PPC comes in on.

Entry points are, in general terms (the client library has not yet been written):

set_direction() sets a GPIO as input, or output.

get_value() returns 0 (low) or 1 (high) as the current status of the pin.

set_value(value) sets the value (if an output) to low (0) or high (1).

configure_irq(type) sets up the hardware to generate an interrupt from pin *pin*. The *type* can be `rising_edge`, `falling_edge`, `level_low`, `level_high`, or `none`, where `none` disables interrupts from this source.

configure(configure_options) Platform dependent interface to set things like drive strength, pull-up or pull-down resistors, whether an output is open-drain, etc.

Some hardware may provide more options; these can be accessed using additional hardware-specific PPC interfaces. The ability to set a GPIO as an interrupt also varies between platforms; some allow any GPIO to be used as in interrupt; others group GPIOs into banks where each bank has a single interrupt that can be allocated to one of the GPIOs in that bank.

At present there is no way to operate on more than one GPIO at a time. This may be added later.

5.9.3 PWM

For the Pulse-Width-Modulation (PWM) class there is a single call that sets period and duty cycle.

5.9.4 Status

These device-class specifications are **subject to change**; code implementing them has not yet been merged.

Chapter 6

Device Discovery

There is **preliminary** work on extracting device parameters from the Linux *device tree* specification as part of configuration tool for the seL4 Microkit [Trustworthy Systems Group, 2023]. This is not yet in a halfway stable state. We will describe this in a later version.

Currently, it is expected that the system designer knows all the devices that will be available and used, and can choose components and assign memory manually to use all the devices that are needed for the system being built.

Chapter 7

Leveraging Linux

Note: This chapter is **preliminary**.

7.1 Component development

The Linux *userspace I/O* (UIO) mechanism [Koch, 2016], designed to support usermode drivers, allows mapping physical memory regions uncached into Linux user space.

We can use this to develop sDDF virtualisers and device drivers in Linux user space, on a Linux system running inside a *virtual machine* (VM) on top of seL4. This allows us to use the normal Linux development tools for component development. While not suitable for performance analysis and tuning, this can simplify and accelerate development.

```
1 chosen {
2     bootargs = "... uio_pdrv_genirq.of_id=sddf_framebuffer ...";
3 };
```

Listing 7.1: Example UIO stub driver command-line configurations.

UIO works with a stub driver in the Linux kernel. There are two generic drivers provided in the mainline Linux source tree, `uio_pdrv_genirq` and `uio_dmem_genirq`. `uio_pdrv_genirq` is used when the device is not a bus-mastering DMA controller; `uio_dmem_genirq` when the device can do DMA. These drivers do not specify which devices they are compatible with; they can be bound to a particular device in various ways. We usually bind at boot-time using a Linux kernel command line argument as shown in Listing 7.1.

```
1 uio@0x30000000 {
2     compatible = "sddf_framebuffer","uio";
3     reg = <0x00 0x30000000 0x00 0x2000000>;
4     interrupts = <GIC_SPI 18 IRQ_TYPE_EDGE_RISING>;
5 };
```

Listing 7.2: Example UIO node in device tree.

To use UIO, a node describing the interface is added to the device tree handed to the virtual machine. For details, see the device-tree specifications in the Linux source tree, Listing 7.2 shows an example.

You can choose the `compatible` string to be whatever does not clash with a real device. The `reg` parameter needs to give the addresses of the regions. When running sDDF (and the development VM) on top of the seL4 Microkit [Trustworthy Systems Group, 2023], this will need to match the address of the memory region specified in the Microkit's *system description file* (SDF), to ensure that Linux and the sDDF component under development see these regions at the same address.

The above example specifies a single 32 MiB region at (virtual-machine) physical address 0x30000000. Likewise, the `IRQ` parameter gives the IRQ controller, IRQ number (relative to the first interrupt on that controller) and the type. Again this must be different from any other interrupt in the device tree.

The virtual-machine monitor (VM) maps each interrupt onto a separate Microkit *channel*. If such a channel is connected to the VM, a notification on that channel (resulting from an interrupt or a notification from another component) will result in injecting the corresponding virtual interrupt into the VM. When the VM re-enables that (virtual) interrupt, this triggers a notification to the Microkit protection domain at the other end of the channel, or an interrupt acknowledgement operation (if the notification was from a real interrupt).

A usermode program running in the VM can then use `mmap()` calls on `/sys/class/uioN/maps/mapn/addr` to access metadata and data regions, `read()` or `poll()` system calls on `/dev/uioN` to wait for notifications, and `write()` on the UIO device file to generate notifications. Here, N identifies the UIO device (of which there can be several), while n refers to the UIO-mapped region of the device (of which there can be multiple as well).

Most of this is abstracted into a (still under development) `libuio` library, making it possible to write components that are relatively easy to port between Linux user space and native Microkit.

In addition, one can prevent the Linux in-kernel driver from binding to a particular device, by changing its `compatible` string in the device tree. One can then bind it to a UIO stub driver and write a user-mode sDDF-style driver that runs in Linux userspace on the guest.

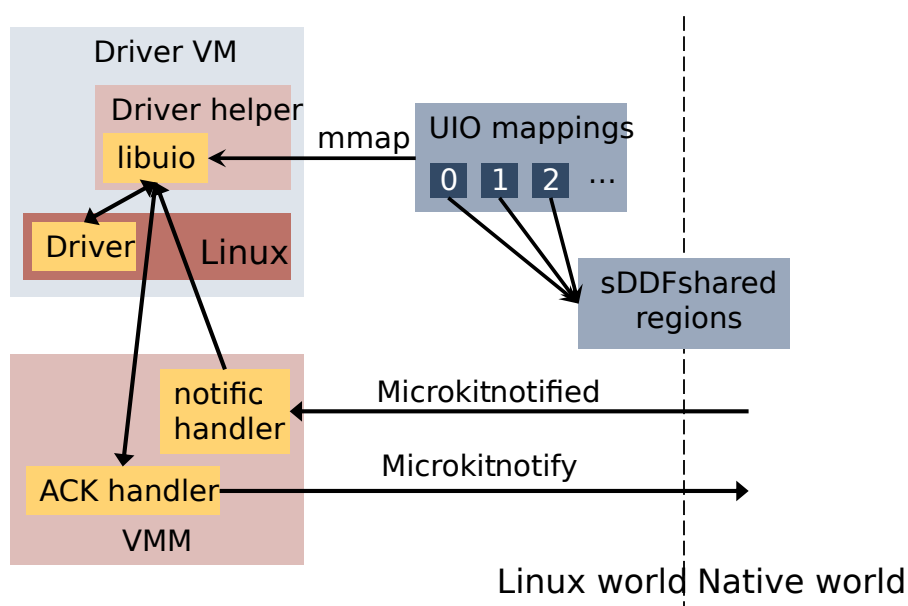


Figure 7.1: Linux driver re-use through UIO.

7.2 Legacy driver reuse

Instead of developing drivers in Linux, we can use the same setup for re-using existing Linux drivers in the sDDF. A virtualiser connects to a “driver PD” that is in reality a Linux VM containing the driver and a userspace program, the *driver helper*, that interacts with the sDDF using the above UIO mechanisms, as indicated in Figure 7.1.

```
1 notified(int channel) {
2     needs_notify = false;
3     while (dequeue_request(request_queue, &request)) {
4         switch (request.storage_command) {
5             case READ_BLOCKS:
6                 llseek(fd, BLOCKSIZE*request.block_number, SEEK_SET)
7                 count = read(fd, adjust_address(request.address),
8                             BLOCKSIZE*request.block_count);
9                 response.count = request.block_count;
10                response.success = count/BLOCK_SIZE;
11                response.result = count == BLOCKSIZE * request.block_count
12                               ? SUCCESS : map_error(errno);
13                enqueue_response(response_queue, &response)
14                needs_notify = true;
15                break;
16                ...
17            }
18        }
19        if (needs_notify)
20            notify(channel);
21    }
```

Listing 7.3: Example of a Linux usermode driver helper for a block device accessed via a regular Linux file (simplified).

The driver helper forwards sDDF requests to the Linux in-kernel driver using standard Linux I/O mechanisms. This can be as simple as reading block-sized chunks from a Linux block device, or even a regular file, using normal Linux read or write calls, as shown in Listing 7.3. The `notified()` function is invoked by `libuio` when the notification channel is signalled. The `notify()` function writes to the appropriate `/dev/uio` special file to cause a notification by the VM on the corresponding channel.

The virtualiser code is unaware of whether the driver is native or a driver VM, the only difference is that the driver metadata region must be configured as *uncached* in the Microkit SDF.

Obviously this approach will not result in high-performance drivers, nor can Linux drivers be trusted, but most drivers are not particularly performance-sensitive, and trust can be minimised by encrypting data that is sent to the device (reducing the potential attacks to denial of service). These limitations are frequently acceptable, particularly if this allows obtaining complex drivers for free.

In such a case, the Linux instance should be reduced to the bare minimum to support the driver in question, as demonstrated many years ago by LeVasseur et al. [2004].

Chapter 8

Security Analysis

Note: This chapter is **preliminary**.

An implication of the threat model of Chapter 2 is that any trusted component of the system must be verifiable. Another implication is that components that interface with untrusted components must sanitise their inputs.

The design of our queues means that pointers into those queues are automatically sanitised (the index is taken modulo the queue size). Similarly, pointers into the data and metadata regions on the client-side of the virtualiser are all offsets into the respective region and only need a check that they do not exceed the region size. These requirements should be easy to verify.

8.1 Trusted components

For now we assume all the components of the sDDF to be trusted, specifically:

- drivers are trusted
- virtualisers are trusted
- copiers are trusted.

In addition, the device hardware is (currently) trusted.

Untrusted device drivers or devices should be achievable, provided that:

- all data handled by the device is encrypted and signed (using higher-level protocols);
- denial of service by the device or driver is not considered a threat;
- the device is not DMA-capable, or its access is restricted by suitable IOMMU mappings (managed by the virtualiser);
- the virtualiser sanitises all references received from the client.

8.2 Verifying components

The driver model is designed to simplify drivers dramatically (compared with e.g. Linux drivers). Drivers are simple state machines free from internal concurrency (but do

access/modify data structures that are concurrently accessed/modified by hardware). In fact, experience with Ethernet drivers shows that they are so simple that little debugging effort is required to achieve correct functionality [Parker, 2023].

The simplicity should make it finally feasible to formally verify drivers for non-trivial real-world devices, such as NICs, although verification requires not only formalising the driver framework but also the device interfaces.

Similar arguments apply to virtualisers: they are also kept simple (even simpler than drivers) and should be verifiable.

The use of virtualisers allows simplifying the IP stack as well, as it only handles packets for a single client and does not have to deal with broadcast requests. Such a simple IP stack may be verifiable as well, and there is work attempting exactly this [Rollins, 2023]. A verified IP stack may eliminate the need for copiers.

When blocking on a response (Notification or IPC reply) from an untrusted component, a trusted component may have to employ a watchdog timer to prevent indefinite blocking, depending on its purpose.

8.3 Verifying component interactions

Simplifying drivers and virtualisers by removing any internal concurrency potentially shifts some of the complexity into the driver framework itself, including concurrency. In fact, our experience shows that the dominating debugging task is not of the driver/virtualiser functionality, but the inter-component signalling protocol. Achieving deadlock freedom is not difficult in itself, but achieving it without having components perform unnecessary signals resulting in other components being awoken without having useful work to perform is the challenge. Optimisations to the simplest policy of "signal each time work has been done" can easily lead to unforeseen deadlocks or delays in processing further down the line, and without the use of an automated tool it can be difficult to calculate by hand whether potential optimisations could lead to a stagnation in the system.

Thus, this is an ideal use case for model checking, and we have developed simplified abstract models of our networking components in order to verify that each optimisation to our signalling protocol cannot lead our system into deadlock. As the signalling protocol is updated during development, we are able to update our abstract models accordingly and rerun them through the verification tool to ensure that we have not inadvertently introduced any issues.

An advantage of the sDDF model is that this complexity is encapsulated in a relatively small code base, and the verification needs to be done only once per device class (and not for each individual driver).

The sanitation requirements on drivers and virtualisers apply to the framework implementation as well, and should be verified.

Chapter 9

Implementation Status

The sDDF is implemented as described above on top of the seL4 Microkit [Trustworthy Systems Group, 2023]. The Microkit simplifies the implementation (among others) by moving the handler loop (see Listing 4.5) into the framework, so the driver and virtualiser implementations mostly consist of the notification handler functions.

The Microkit implementation presently provides high-performance networking (see Chapter 10). There are a implementations for a number of other device classes (serial, I2C, storage, graphics, audio) which are still in a preliminary state. The framework is presently based on active virtualiser and driver components.

The code accessible on GitHub under a BSD license: <https://github.com/au-ts/\gls{sddf}>.

Chapter 10

Performance

10.1 Performance evaluation setup

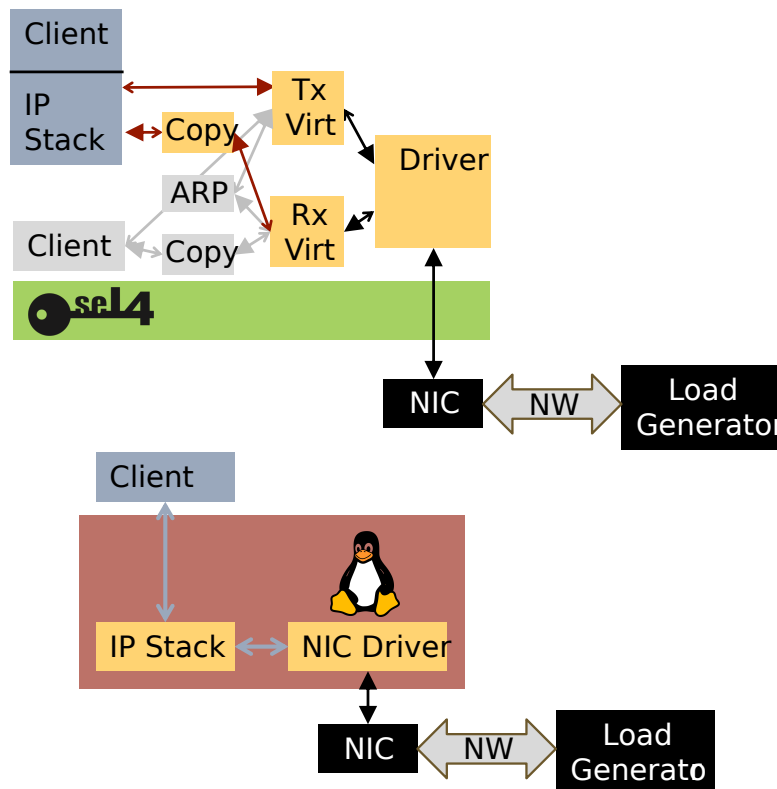


Figure 10.1: Evaluation setup for sDDF (top) and Linux (bottom).

We evaluate sDDF networking performance against the older CAMkES-based driver framework, as well as against Linux.

The evaluation platform is an Arm Cortex-A53 processor based on the i.MX8M Mini microarchitecture running at 1.8GHz, equipped with a Gigabit Ethernet NIC using a 10/100/1000 Atheros AR8031 PCM. We run *ipbench* [Wienand and Macpherson, 2004] for distributed load generation from four client machines, each sending UDP packets to the seL4-based target machine, which sends them through an IP stack (*lwIP* [Dunkels, 2001]) to

a co-located client, which returns the packets, slightly modified according to the ipbench protocol. The load generators count the successful replies to determine the achieved throughput and latency, while a low-priority thread on the target machine measures idle time to determine the CPU load imposed by the test.

Figure 10.1 shows the setup. The sDDF system is based on the seL4 Microkit [Trustworthy Systems Group, 2023]. Linux benchmarks use a Linux 6.1.1 kernel system on the same hardware.

10.2 Performance results

10.2.1 Simplified system, CAMkES, Linux

We initially evaluate a simplified two-component system, where the client, IP stack and cache management are merged into a single component (no device sharing supported), and only the driver is separate. This setup, while not very realistic, is directly comparable to the older driver framework based on CAMkES [Kuz et al., 2007]. We also convert the CAMkES framework to our transport layer and run it in the same, two-component configuration.

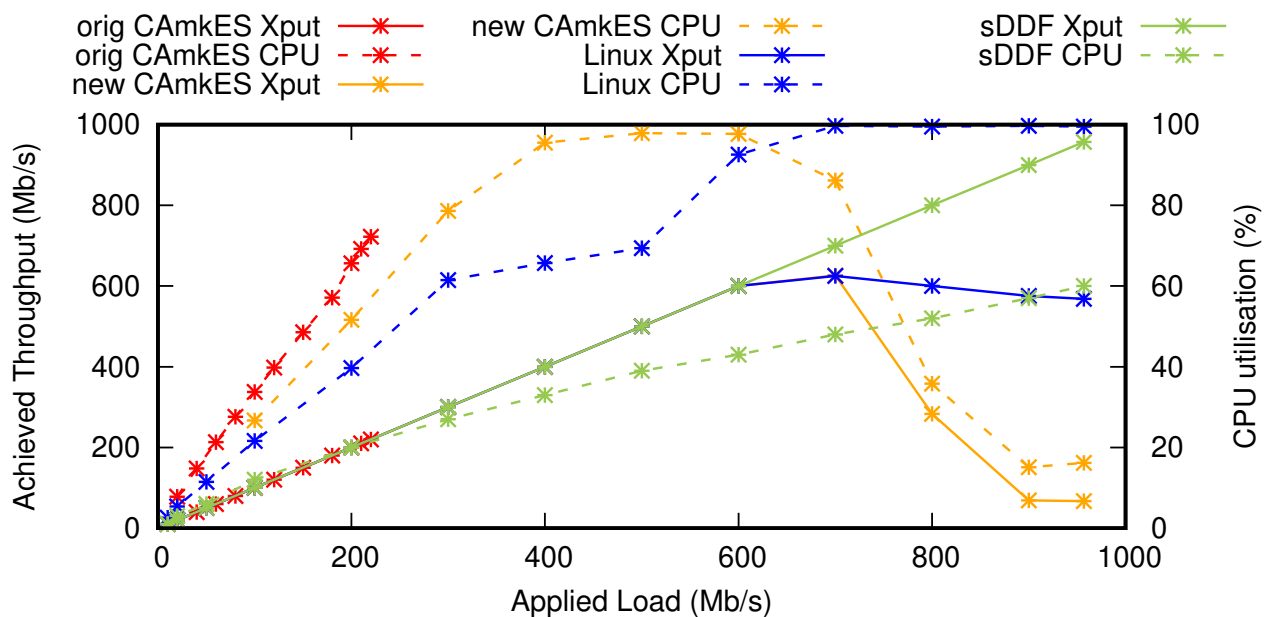


Figure 10.2: sDDF networking performance, compared to original CAMkES, CAMkES with the new transport layer, and Linux. All benchmarks use a single core.

Figure 10.2 shows the results. We can see that the original CAMkES system already reaches 66% CPU utilisation at 200 Mb/s applied load, and can be expected to saturate the CPU at around 350 Mb/s.¹

CAMkES adapted to the new transport layer (“new CAMkES”) uses somewhat less CPU (52% vs. 66% at 200Mb/s, a 20% improvement) but maxes out the CPU above a load of 400 Mb/s. Achieved throughput scales with applied load to 600 Mb/s, and then collapses when applied load exceeds 700 Mb/s. Performance collapse under overload is not untypical in network systems, and generally results from latency blowout.

¹The measurements on “original CAMkES” (in fact after a fair amount of performance tuning) were done in 2021 and we only have records for the low-load cases.

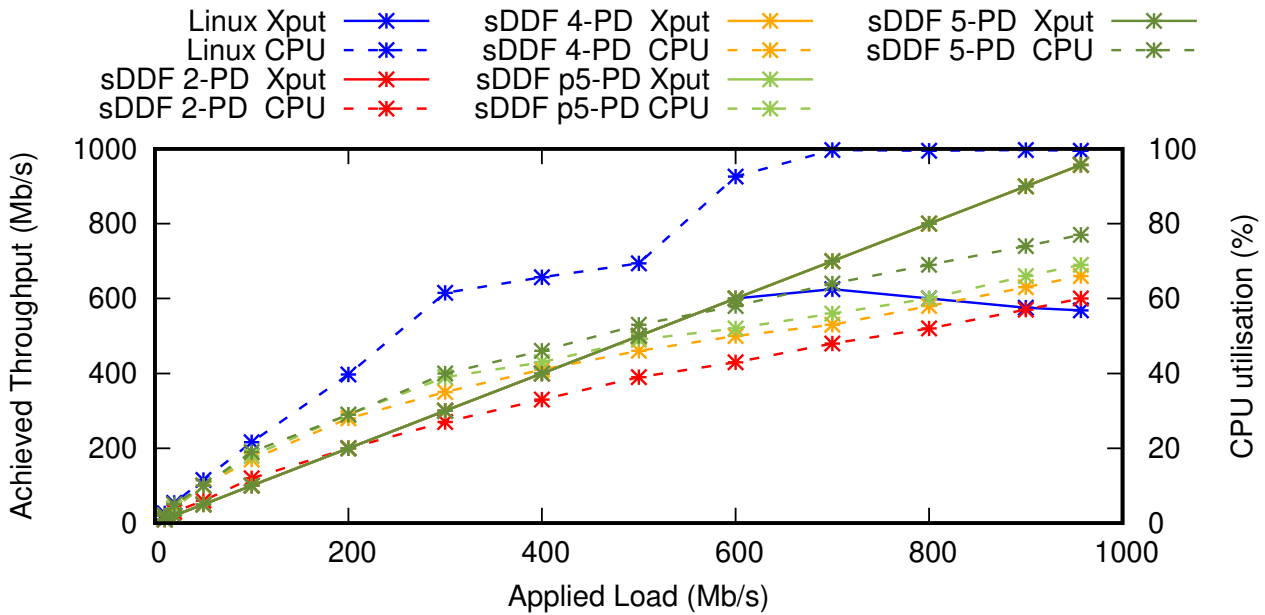


Figure 10.3: Performance of a two-, four-, pseudo-five- and a standard five-PD sDDF system compared to Linux, all on a single core.

Linux behaves somewhat better, also scaling just over 600 Mb/s, when it saturates the CPU. While nowhere near as extreme as the performance collapse of CAMkES, Linux throughput still degrades somewhat under overload.

In contrast, the sDDF system scales linearly to the wire speed.² It does *not* max out the CPU, but handles the full applied load with only 60% CPU utilisation, leaving plenty of performance head space. At all load levels it uses far less CPU than Linux. At low load (100 Mb/s), the sDDF CPU usage is 12%, beating all the other systems by a factor of 1.9 or more. The ratio becomes smaller at higher load due to batching.

10.2.2 Cost of modularity and security

Now we compare the simplified two-PD system to the more realistic one of Figure 10.1, where five PDs (out of a total of 8) are involved in handling each packet, *and* incoming data is copied to separate client address spaces. For more analysis, we present also two intermediate systems: a four-PD system, which has the Copier component removed, and a pseudo-five-PD system (labelled “p5-PD”) where instead of a copier we have a dummy component that just forwards packets without doing any meaningful processing.

Figure 10.3 shows the results. We can see that the three extra context switches plus one data copy increase CPU usage noticeably, from 12% to 19% at 100 Mb/s, and from 60% to 77% at high load, a relative increase of 64% at low load and 27% at high load (again, the effects of overheads are lessened by batching). Yet, this highly modular setup still clearly outperforms Linux by more than a factor 1.5 (low load).

The intermediate systems show that the extra data copy plays a similar role as the extra context switches, and they all add low to moderate overheads. This becomes clearer in Figure 10.4, which compares per-packet processing costs. It clearly shows that the difference

²Note that a Gigabit Ethernet NIC can handle about 950 Mb/s of payload with standard 1.5 KiB Ethernet frames, the rest of the bandwidth is lost to Ethernet headers and inter-packet gaps.

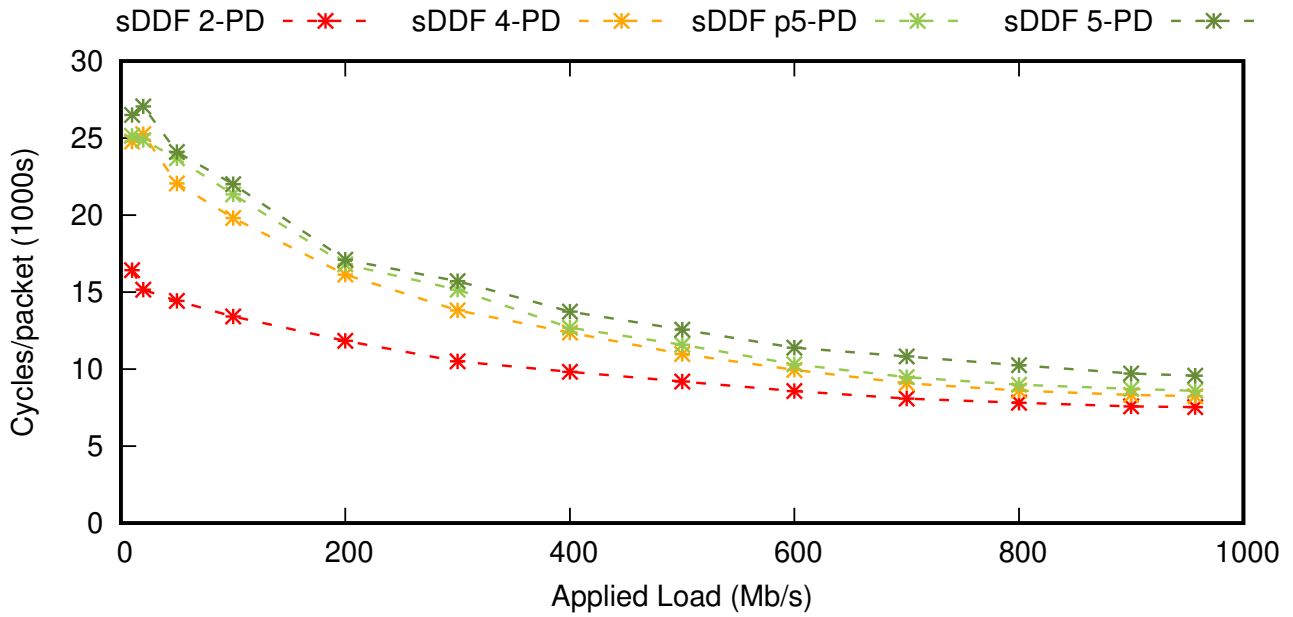


Figure 10.4: Comparing per-packet processing costs of the four sDDF systems.

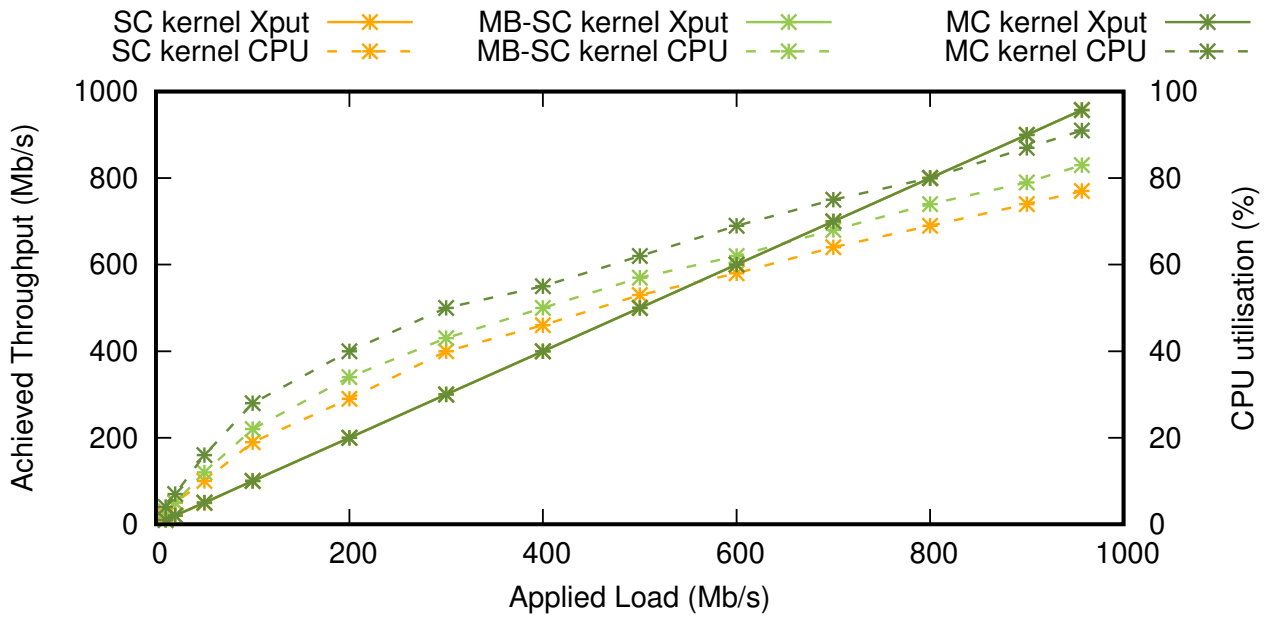


Figure 10.5: Single-core sDDF performance using single core seL4 kernel with and without memory barriers enabled vs multicore seL4 kernels.

between the 4-, pseudo-5- and 5-PD systems is fairly small: the extra PD (without copying) increases per-packet processing cost by 1.5k cycles at 100 Mb/s and 350 cycles at wire speed, a relative increase of 8% and 4% respectively. In comparison, copying adds 700 cycles at low and 1,000 cycles at high load, a relative increase of 3% and 11% respectively.³

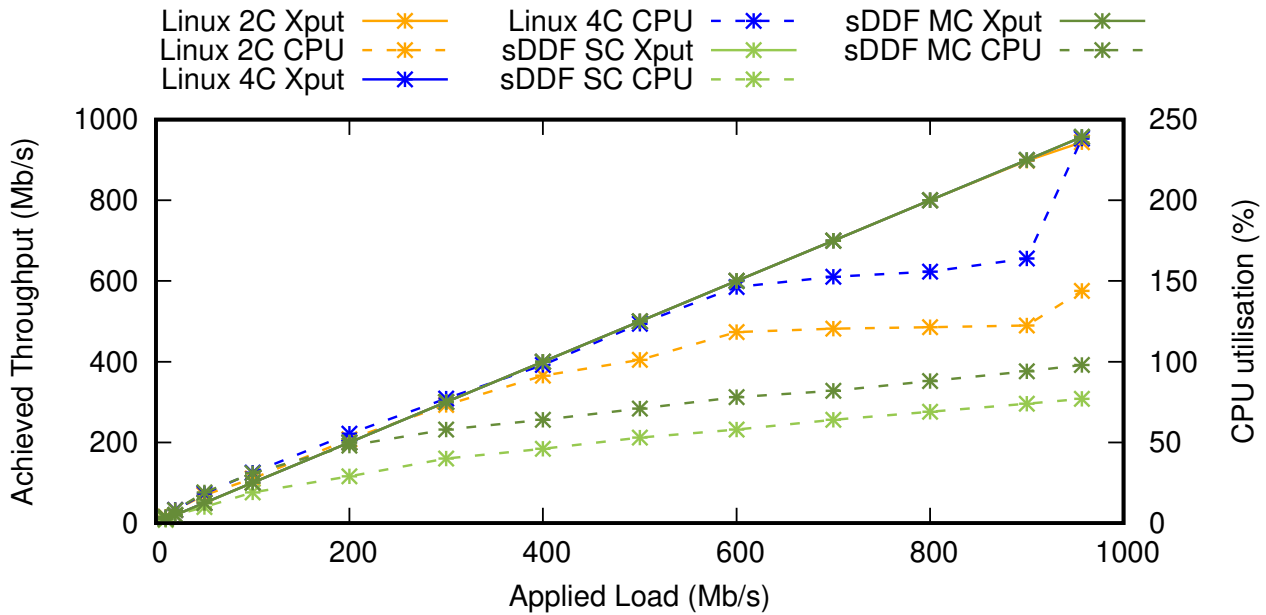


Figure 10.6: Performance of sDDF running single-core (sDDF SC) vs. distributed over two cores (sDDF MC) compared to Linux running on two (Linux 2C) or four (Linux 4C) cores.

10.2.3 Multicore

All the above benchmarks were run on a single core, using an system that is optimised for that case: the seL4 kernel compiled with multicore support disabled, and no memory barriers in the sDDF queue libraries.

Enabling multicore increases the baseline system-call cost of seL4 by about 50%, basically the cost of acquiring and releasing the kernel lock. Additionally, supporting a multicore system requires memory barriers when enqueueing or dequeuing entries in the shared queues, to ensure they work correctly of the other PD accessing the queue runs on a separate core.

The CPU load figures in Figure 10.5 show that both requirements add similar overheads. Here we compare the strict single-core configuration used so far (“SC kernel”) with a version that adds the memory barriers but still uses the single-core kernel (“MB-SC kernel”), and additionally the same sDDF version running on seL4 with multicore enabled (“MC kernel”). Adding memory barriers adds about 15% overheads at low load and 8% at high load, while the total cost of enabling multicore is almost 50% at low load and almost 20% at high load.

From now on we use the kernel and sDDF version that fully supports multicore. We compare multicore-enabled sDDF running on a single core with sDDF with its components distributed over two cores: the driver and Tx virtualiser run on one core, and the other components, Client, Copier and Rx virtualiser, on the other. We compare to two Linux configurations, one restricted to two cores and the other giving it free choice of all four cores.^c

The results are in Figure 10.6. Having 2 cores at its disposal, Linux just manages to handle the applied load (as indicated by latencies, which increase significantly at the highest applied load). Its CPU usage is well above that of the sDDF system. On four cores, Linux shows significantly higher overall CPU load, while still barely coping with the network load (indicated by the CPU usage blowing out at 950 Mb/s, and also reflected in a latency blowout).

^cOne would expect the absolute per-packet copying cost to be independent of load. We did not investigate this further, but suspect caching effects.

In contrast, the sDDF system still has significantly lower CPU utilisation than Linux, total CPU load staying (just) below 100% of one core even at the highest network load. The total CPU load of the two-core configuration is 65% higher (31% vs 19%) than the single-core version at low load and 30% higher (98% vs 77%) at high load. This is due to cross-core notifications being more expensive than intra-core notifications.

The overall CPU load of an sDDF system will obviously be sensitive to the allocation of PDs to cores, which, in turn, depends on other considerations, such as maximising cache locality or freeing up cores for other activities. The optimal configuration will therefore be dependent on the use case. The multicore-enabled configuration provides full location transparency to PDs.

10.3 Discussion

The framework, running on the Microkit, clearly over-achieves our goal of “performance comparable to Linux”, outperforming Linux by a significant margin under all configurations evaluated. Performance is vastly better than that of the CAMkES system adapted to our transport layer (which is already a significant improvement over the performance of the previous CAMkES-based driver framework).

The performance comparison to Linux is especially encouraging, given that the Linux IP stack is supposedly well optimised (especially for multicore use), while we are using the simple lwIP [Dunkels, 2001]. We find that even on multicore (the scenario most favourable to Linux) our per-packet costs are significantly lower.

Parker [Parker, 2023] conducted a more comprehensive performance analysis, including highly asymmetric (send-mostly or receive-mostly) traffic, multiple active clients with traffic shaping, more multicore configurations, and forcing the system to be overloaded. The evaluation confirmed that the sDDF/Microkit system shows no performance collapse under any overload conditions evaluated, unlike what we observed in Section 10.2.1 for the CAMkES system.

The evaluation, specifically the results shown in Section 10.2.2, provide strong justification for our highly modular design: The cost of a PD switch is small compared to the base processing cost.

This confirms the original hypothesis driving the sDDF design: Context-switching costs (of the order of 400–500 cycles for the single-core kernel on our Armv8 platform, about 50% higher for the multicore kernel, and still higher for cross-core notifications) are not a major factor. In fact, we measure handling costs of less than 50,000 cycles per packet on sDDF under any conditions, and no more than about 12,000 cycles under high network load. Hence, the pure kernel overhead of adding an extra component (two system calls) to the packet-processing pipeline adds no more than about 15% (usually much less) to the handling cost.

As such, our main take-away is that the fine-grained modularisation, resulting from the strict separation of concerns, works and adds far less overhead than the complexity of the Linux system. The reason is two-fold:

1. as stated, context-switching costs in seL4 and the Microkit are small enough to not significantly impact performance, and
2. the sDDF model encourages implementation simplicity that can *reduce* overheads (which is the reason sDDF outperforms Linux).

The latter has further, highly encouraging implications: We can keep individual component implementations very simple, without sacrificing performance. This simplicity not only greatly reduces debugging efforts, it should also make automatic verification techniques applicable. The experience with verifying the Microkit [Paturel et al., 2023] gives us confidence that this should be achievable.

The evaluation of the multicore configurations are also highly encouraging: It shows that the sDDF naturally distributes across cores. While all PDs are strictly sequential (resulting in decreased code complexity), the design makes them location-transparent, meaning they can be arbitrarily allocated to cores in order to balance load. The evaluation confirms the feasibility of this design choice of keeping components sequential and instead allowing arbitrary concurrency *between* components.

Further performance improvements seem possible with improvements to the kernel itself. Candidates are fast-pathing notifications to higher-priority threads, and taking notifications out of the kernel lock.

Chapter 11

Conclusions

We have developed a low-overhead device driver framework for seL4, sDDF, implemented it on the seL4 Microkit, and comprehensively evaluated it on network devices. It is based on a highly efficient, asynchronous zero-copy transport layer using lock-free single-producer, single-consumer, bounded queues.

Measured performance shows a vast improvement over the CAMkES-based framework, as well as significantly better performance than Linux, despite the higher number of system calls and context-switches required. In particular we find that a highly componentised sDDF system easily distributes across cores, while each individual component being strictly sequential, reducing its complexity.

So far we have only examined the performance of the network system. We have started applying the same ideas to other device classes, especially storage. Other device classes are generally much less sensitive to system performance than networking, so we do not expect any surprises there.

We are also working on integrating sDDF with the Microkit's virtualisation infrastructure, which will support VMs as clients as well as drivers and virtualisers. On the one hand, this provides a convenient way of developing sDDF components as Linux programs that can later be converted into native sDDF PDs. On the other hand, this allows re-use of existing Linux drivers in the form of a driver-VM.

Obviously, such legacy drivers are in no way trustworthy. Furthermore, virtualisation overheads are expected to dominate sDDF overheads, so these configurations are important for functionality (especially the ability to re-use unmodified Linux drivers) but will not be suitable for performance-sensitive devices.

Bibliography

- Adam Dunkels. Minimal TCP/IP implementation with proxy support. Technical Report T2001-20, SICS, February 2001. <http://www.sics.se/~adam/thesis.pdf>. 48, 53
- Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *ACM Symposium on Operating Systems Principles*, pages 207–220, Big Sky, MT, USA, October 2009. ACM. URL https://trustworthy.systems/publications/nicta_full_text/1852.pdf. 1
- Hans-Jürgen Koch. The userspace I/O HOWTO, 2016. URL <https://www.kernel.org/doc/html/latest/driver-api/uio-howto.html>. Retrieved 2024-2-27. 42
- Hermann Kopetz. The time-triggered architecture. *Proceedings of the IEEE*, 91:112–126, 2003. 23
- Ihor Kuz, Yan Liu, Ian Gorton, and Gernot Heiser. CAmkES: A component model for secure microkernel-based embedded systems. *Journal of Systems and Software Special Edition on Component-Based Software Engineering of Trustworthy Embedded Systems*, 80(5): 687–699, May 2007. URL <http://dx.doi.org/10.1016/j.jss.2006.08.039>. 49
- Ben Leslie, Peter Chubb, Nicholas FitzRoy-Dale, Stefan Götz, Charles Gray, Luke Macpherson, Daniel Potts, Yueting (Rita) Shen, Kevin Elphinstone, and Gernot Heiser. User-level device drivers: Achieved performance. *Journal of Computer Science and Technology*, 20(5):654–664, September 2005. URL https://trustworthy.systems/publications/papers/Leslie_CFGGMPSEH_05.pdf. 8
- Joshua LeVasseur, Volkmar Uhlig, Jan Stoess, and Stefan Götz. Unmodified device driver reuse and improved system dependability via virtual machines. In *USENIX Symposium on Operating Systems Design and Implementation*, pages 17–30, San Francisco, CA, US, December 2004. 44
- Lucy Parker. High-performance networking on seL4. BSc(Hons) thesis, School of Computer Science and Engineering, Sydney, Australia, November 2023. 23, 46, 53
- Mathieu Paturel, Isitha Subasinghe, and Gernot Heiser. First steps in verifying the seL4 Core Platform. In *Asia-Pacific Workshop on Systems (APSys)*, Seoul, KR, August 2023. ACM. URL https://trustworthy.systems/publications/papers/Paturel_SH_23.pdf. 54
- Wyeth Greenlaw Rollins. Toward a verified, minimal IPv6 network stack implementation, September 2023. URL <https://sel4.systems/Foundation/Summit/2023/abstracts2023#a-toward-verified>. Talk at the seL4 Summit. 46

- Mark Rutland. Stale data, or how we (mis-)manage modern caches, 2016. URL http://events17.linuxfoundation.org/sites/events/files/slides/slides_17.pdf. 17
- Leonid Ryzhyk, Peter Chubb, Ihor Kuz, and Gernot Heiser. Dingo: Taming device drivers. In *EuroSys Conference*, pages 275–288, Nuremberg, DE, April 2009. URL https://trustworthy.systems/publications/nicta_full_text/1527.pdf. 7, 8, 15
- Leonid Ryzhyk, Yanjin Zhu, and Gernot Heiser. The case for active device drivers. In *Asia-Pacific Workshop on Systems (APSys)*, pages 25–30, New Delhi, India, August 2010. URL https://trustworthy.systems/publications/nicta_full_text/3681.pdf. 16
- seL4 Foundation. The seL4 microkernel, 2021. URL <https://sel4.systems/>. 1
- Trustworthy Systems Group. The seL4 Microkit, October 2023. URL <https://trustworthy.systems/projects/microkit/>. 41, 43, 47, 49
- Ian Wienand and Luke Macpherson. ipbench: A framework for distributed network benchmarking. In *Conference for Unix, Linux and Open Source Professionals (AUUG)*, pages 163–170, Melbourne, Australia, September 2004. URL https://trustworthy.systems/publications/papers/Wienand_Macpherson_04.pdf. 48
- Wikipedia. Self-monitoring, analysis and reporting technology, 2004. URL https://en.wikipedia.org/wiki/Self-Monitoring,_Analysis_and_Reporting_Technology. 33

Appendix A

OS Abstraction

The sDDF is meant to be usable by all seL4-based OSes. As such it needs an abstraction layer for invoking underlying OS services. This abstraction layer consists of a number of (inlineable) functions that map the mechanisms required by the sDDF to the underlying OS.

A.1 Microkit Mapping

The seL4 Microkit provides a small number of abstractions:

- a *PD* is an address space plus a thread of control. It has a name, and a priority for its thread; in addition its budget can be controlled,
- a *channel* is a small integer that is passed with a notification, to identify what kind of notification it is
- a *notification* is an asynchronous signal between PDs
- a *PPC* is a synchronous signal, that can pass values in a number of *message registers*.

The sDDF mechanisms map one-to-one onto these abstractions.

A PD has up to three entry points. Its `init()` entry point is called when the PD first runs; after `init()` returns, the `notified(channel)` entry point is invoked whenever the PD receives a notification from another PD or from an interrupt source. The `protected(channel)` entry point is invoked from a PPC call from another PD.

```

1  /* Return the name of the currently running protection domain */
2  char *sddf_get_pd_name();
3
4  /* Acknowledge an interrupt */
5  void sddf_irq_ack(sddf_channel ch);
6
7  /* Notify a channel */
8  void sddf_notify(sddf_channel ch);
9
10 /* Generate a notification, but not until this PD is about to sleep */
11 void sddf_deferred_notify(sddf_channel ch);
12
13 /* Acknowledge an interrupt, but not until this PD is about to sleep */
14 void sddf_deferred_irq_ack(sddf_channel ch);
15
16 /* Return any outstanding deferred notification */
17 unsigned int sddf_deferred_notify_curr(void);
18
19 /* Invoke the _protected_ entrypoint in another PD */
20 microkit_msginfo sddf_ppcall(sddf_channel ch, microkit_msginfo msginfo)
21
22
23 /* Get or set a (seL4) message register */
24 uint64_t sddf_get_mr(unsigned int n);
25 void sddf_set_mr(unsigned int n, uint64_t val);

```

Listing A.1: sDDF interface functions.

Appendix B

Overview of Changes

B.1 Changes For Release 0.7 of 2025-03-21

Updates to the I2C class following a redesign Section 5.3.1.

B.2 Changes Since Release 0.6 of 2025-03-21

B.2.1 New device classes

We added device classes for sensors, PWM, and fan in Section 5.9, and a pinmux driver description in Section 5.8.

B.3 Changes Since Release 0.4 of 2024-03-27

B.3.1 OS abstraction

We made the sDDF largely independent of the underlying OS by providing an OS abstraction layer (Appendix A). We specify the mapping to the Microkit in Section A.1.

B.3.2 More device classes

We added the I2C class, see Section 5.3.1.

B.4 Changes Since Release 0.2 of 2022-10-03

The document has undergone major revision and restructure, with changes in many places. We summarise the main changes.

B.4.1 More device classes

While networking is still the running example of the bulk of the document, we tried to point out where other device classes may differ.

We moved device-specifics into a separate chapter, provide details of some device classes, and added place-holders for others.

B.4.2 Mandatory virtualisers, cache maintenance

We generalised the concept of multiplexers to virtualisers, as they are also responsible for translating between client virtual and I/O addresses. Consequently, virtualisers are now mandatory components (required even if a device only has a single client).

We include a discussion of cache maintenance, and specify that this is the responsibility of the virtualisers.

B.4.3 Completely separated Tx and Rx paths

We require that for device classes providing spontaneous input (e.g. networks) the Tx and Rx paths are completely separated. This means separate Tx/Rx virtualisers, and separate Tx/Rx data and metadata regions.

B.4.4 Clarified terminology

We removed the term “ring buffers” as this caused confusion with data buffers. Instead we consistently use the term “queue” for the metadata structures and reserve “buffer” for the actual I/O-data locations.

We now require the transmit, receive, request metadata regions as well as the virtualisers (formerly: multiplexers) to be disjoint (before this was considered an option).

We removed the concept of a “Server” in the driver model, there are only drivers, virtualisers and “clients” (the latter possibly consisting of a pipeline of components).

We use the term *metadata region* for all software-defined regions shared between components (other than the data regions). We explicitly distinguish the device’s *control region* (i.e. the memory-mapped device registers) from its metadata region.

We clarified (and fixed!) the concept of *ownership* of I/O data and metadata, to aid verification.

B.4.5 Linux-based component development and legacy re-use

We added Chapter 7 which describes how to leverage UIO to do driver development under Linux, and integrate legacy Linux drivers.

B.4.6 Performance evaluation

We extended and updated the performance evaluation, including extending to multicore scenarios.

We furthermore found that the Linux distribution we used to benchmark against had serious performance deficiencies which do not exist in the mainline kernel, so we re-ran Linux evaluations with a version built from source.

We extended the performance discussion as a result.

In general, performance is significantly improved compared to the original release, and keeps out-performing Linux.

B.4.7 Changes resulting from evaluation

We changed/refined details of the Ethernet driver interface as the result of evaluating implementation practicalities and performance.

We also reported on other implementation/evaluation experience, including debugging and the use of model checking of the signalling protocols.